

Complexity Is Not an Edge

Complexity Is Not an Edge: An Empirical Study of Machine-Learning Arbitrage on Kalshi and Polymarket

Ian Sabia (U33871576), Alvin Jang (U64760665) Department of Data Science, Boston University DS340 — Spring 2026 Final Project April 27, 2026

Abstract

We study cross-platform price discrepancies between **Kalshi** (CFTC-regulated event contracts) and **Polymarket** (on-chain prediction market) and ask whether increasing model complexity improves arbitrage detection. We build an end-to-end system that matches contracts via sentence embeddings plus a 10-rule structural quality filter, engineers 59 features (13 market-microstructure), and trains four model tiers — regression baselines (LR, XGBoost), sequence models (GRU, LSTM), PPO, and PPO+autoencoder — under one evaluation protocol. Across five regimes (single-split, 11-window walk-forward, category-stratified, data-scaling, live paper trading) the **simplest models consistently dominate**. On 6,802 train / 1,673 test rows across 144 matched pairs at \$100 position size, **Linear Regression achieves a per-trade Sharpe of 0.501 with +15.0 bps per-trade alpha**, tied with XGBoost (0.499 / +14.9 bps), versus 0.473 / +14.3 bps for LSTM, 0.459 / +14.0 bps for GRU, and **+0.5 bps for PPO+autoencoder** (Table 2). Every walk-forward window is profitable for every ML model. A pre-submission audit (Phase 18, `AUDIT_REPORT.md`) found 142 embargo violations in the original row-index split; retrained leakage-free on a pair-stratified split (115/29 pairs), **per-trade Sharpe rises to 0.516 with +15.7 bps alpha** (drift +2.99%, within bootstrap CI) — the per-trade edge is robust. The per-pair annualized Sharpe is regime-dependent (\$5.8). The central empirical answer: **at this data scale, complexity is a liability, not an edge**. The alpha lives in the matching pipeline and the oil/commodities asset class (live-validated in Phase 15; see §5.9) — not in the models.

Keywords: prediction markets, arbitrage, market microstructure, XGBoost, LSTM, PPO, walk-forward validation, simplicity.

1. Introduction

1.1 Problem Statement

Prediction markets allow traders to buy and sell contracts that pay out \$1 if a specified real-world event occurs (e.g., “Will CPI inflation exceed 3.0% in May 2026?”) and \$0 otherwise. The equilibrium price of such a contract is interpreted as the market-implied probability of the event. Two large U.S.-accessible venues — **Kalshi** (CFTC-regulated, operating since 2021) and **Polymarket** (on-chain via Polygon, operating since 2020) — frequently list contracts that reference the *same* underlying event but trade at materially different prices. These cross-platform discrepancies can persist for hours or days, representing a potential statistical-arbitrage opportunity if they can be detected, matched correctly, and traded before they close.

The central research question of this project is:

Does increasing model complexity improve arbitrage detection in cross-platform prediction markets, and if so, when is that complexity justified?

Answering this question rigorously requires four things, each of which we provide:

1. A *shared evaluation protocol* across model families of very different complexity.
2. A *robust matching pipeline* — without it, models are fitting noise.
3. *Multiple independent evaluation regimes* (single-split, walk-forward, scaling curve, live trading) so conclusions do not hinge on a single train/test split.
4. An *honest accounting* of transaction costs, Sharpe inflation, and survivorship bias.

1.2 Motivation

Two distinct audiences motivate this work:

Academic. A January 2026 working paper (arXiv 2601.07131) argues that “Matched Filter” normalization grounded in market-microstructure theory captures virtually all exploitable signal for investor-flow prediction, and that feature engineering consistently beats deep learning in this regime. Prediction-market arbitrage is a natural testbed for that claim because contracts have a finite lifetime, bounded

payoffs, and deterministic settlement — eliminating much of the noise that hides signal in equities. If the “simple beats complex” result holds in this cleaner domain, it is strong evidence for the broader thesis.

Applied. Prediction markets are growing rapidly: Polymarket processed over \$1 billion of volume during the 2024 election cycle, and Kalshi added sports-event and economic-indicator contracts in 2025. As liquidity grows, cross-platform inefficiencies become a relevant feature of the ecosystem. Understanding which modeling approaches genuinely add value — and which merely add training cost — is a practical question for market participants.

1.3 Background

Prediction-market mechanics. A binary event contract on platform P at time t has a market price $p_P(t) \in [0, 1]$ interpretable as the implied probability of the event. At event resolution, the contract pays exactly \$1 or \$0. For two platforms A, B listing the *same* event, the **spread** is $s(t) = p_A(t) - p_B(t)$. If the contracts are genuinely equivalent, $s(t) \rightarrow 0$ as t approaches resolution, modulo fees and basis risk. Our models predict the one-step change $\Delta s(t) = s(t+1) - s(t)$.

Why discrepancies exist. Kalshi and Polymarket have disjoint user bases (Kalshi: retail U.S. traders, regulated brokerage experience; Polymarket: crypto-native, self-custodial wallet), different fee structures (Kalshi: \$0.00–\$0.07 per contract; Polymarket: 0% trading fee, \$0.01–\$0.05 Polygon gas), and different listing policies (Kalshi has contract-expiry time-windows as granular as one hour; Polymarket tends to list monthly or quarterly contracts). These differences slow cross-platform arbitrage and sustain price dispersion.

Cross-platform matching is non-trivial. There are no shared identifiers between platforms; the same event may be titled “Will the Fed cut rates at the March 2026 meeting?” on Kalshi and “March Fed decision” on Polymarket with different explanatory text. Contract resolution criteria can also differ in subtle ways (e.g., “who wins the 2026 NBA Finals” vs. “who wins the 2026 NBA MVP”). Semantic matching — with a principled quality filter — is therefore a first-class concern, not a preprocessing afterthought.

1.4 Contributions

This paper makes the following contributions:

1. **A functioning end-to-end arbitrage system**, from data ingestion through live paper-trading on BU SCC, with redundant deployment on GitHub Actions. At its peak, the system monitored **11,582 matched contract pairs**.
2. **A complexity-vs-performance benchmark** comparing four tiers of models under an identical evaluation protocol, evaluated across five independent regimes.
3. **An 11-window walk-forward validation** (Fig. 1) demonstrating the edge is stable and *improving* over time (per-trade Sharpe 0.31 $\rightarrow$ 0.53, every window profitable for every ML model).
4. **A per-category stratified analysis** (Table 3) showing that the “XGBoost wins overall” conclusion is driven by specific regimes (inflation, crypto), not universal model superiority — a nuanced finding.
5. **A six-point data-scaling curve** (Table 4) directly addressing the “would more data flip the ranking?” question — it does not.
6. **A transaction-cost sensitivity analysis** (Table 7) showing the system remains profitable at all realistic fee levels from 0 pp to 7 pp.
7. **An honest Sharpe-ratio accounting** (§5.8, Table 8) leading with **per-trade Sharpe of 0.501 (leaky) $\rightarrow$ 0.516 (leakage-free purged split)** and per-trade alpha of **+15.0 $\rightarrow$ +15.7 bps** at \$100 position size — the load-bearing headline that survives the Phase 18 embargo audit. The per-pair annualized Sharpe is regime-dependent and reported as a secondary statistic with full caveats; we discuss this explicitly in §6.4.
8. **A negative result:** reinforcement learning (PPO, PPO + autoencoder) is *worse* than every other model in our evaluation. We report this transparently rather than suppressing it.

2. Related Work

Prediction-market efficiency. Manski (2006) and Wolfers & Zitzewitz (2004) established the foundational result that prediction-market prices, while not fully efficient, are among the best available probability forecasts for well-specified events. Cross-platform dispersion has been less studied — most prior work treats a single venue in isolation.

Favorite-longshot bias. A robust empirical regularity in prediction markets is that longshot contracts (low prices) trade *above* their realized probabilities while heavy favorites trade *below* their realized probabilities. Burgi, Tuccella & Zitzewitz (2026) quantify this bias on both Kalshi and Polymarket and show it differs across platforms — creating a structural, model-free source of cross-platform spread. We incorporate this directly as a feature (§3.3).

Market-microstructure features. Amihud (2002) introduced the illiquidity ratio $|r|/V$ as a near-universal illiquidity measure. Corwin & Schultz (2012) derived a closed-form estimator of the effective bid–ask spread from daily high-low prices. Kyle (1985) introduced λ as the price-impact coefficient in informed-trading models. Roll (1984) derived an implied spread from return autocorrelation. These features are standard in equity microstructure but, to our knowledge, have not been systematically applied to prediction-market spread prediction.

Feature engineering vs. deep learning. A persistent finding in applied ML is that on tabular data with moderate sample size, gradient-boosted trees (XGBoost, LightGBM) match or beat deep neural networks (Grinsztajn, Oyallon & Varoquaux, NeurIPS 2022). The January 2026 paper we cite above extends this to time-series investor-flow prediction. Our results are consistent with both.

3. Data and Feature Engineering

3.1 Data Sources

Kalshi exposes a public REST API (`api.elections.kalshi.com/trade-api/v2`) that requires no authentication for market-data endpoints. We fetch hourly OHLCV candlesticks (`period_interval=60`) and market-metadata endpoints for active and historical markets. A critical implementation detail: Kalshi splits market history at a roughly three-month cutoff, with older markets requiring a different `/historical/` endpoint path; we query `/historical/cutoff` to determine which endpoint applies per ticker.

Polymarket is significantly more complex, exposing *three* separate APIs: **Gamma** (market metadata), **CLOB** (order-book and price-history for active markets), and the **Data API** (fills and trades for resolved markets). Polymarket uses opaque numeric token IDs rather than slugs, so we perform a two-stage lookup: Gamma gives us `clobTokenIds`, which then key the CLOB price queries. A non-obvious gotcha: Gamma’s singular `condition_id=` query parameter returns *unrelated random markets* (we observed “Russia–Ukraine ceasefire before GTA VI?” returned for a Canadian-recession query), while the plural `condition_ids=` parameter returns the exact match. Additionally, the `/prices-history` endpoint returns empty data for *resolved* markets, forcing us to reconstruct historical prices from trade records via the Data API.

3.2 Matching Pipeline

Semantic matching uses `sentence-transformers/all-MiniLM-L6-v2` to embed the concatenated title + description of each market on each platform into a 384-dimensional vector. We compute cosine similarity between all pairs via normalized dot-product matrix multiplication. An initial $O(N \cdot M)$ keyword pre-filter — which would have taken ~ 6.6 hours on our universe — was replaced with the matrix approach that runs in ~ 80 seconds, a $300\times$ speedup.

Semantic similarity alone is insufficient: the embeddings cluster “NBA Finals winner” and “NBA MVP” very close together. We layered **10 structural quality-filter rules** on top (`src/matching/quality_filter.py`):

1. Sports: wins-vs-champion mismatch (e.g., “Lakers win Game 4” vs “Lakers win the title”).
2. Fed: meeting-month mismatch (March vs. May).
3. Politics: cabinet-confirmation vs. cabinet-nomination.
4. Commodities: state-specific (CA/FL/NY/TX) vs. national (added in April after discovering 135 false matches).
5. Inflation: exact CPI vs. PCE vs. core-CPI disambiguation.
6. Strict expiry-date window: ± 2 days maximum.
7. Symbol-suffix matching: exact strike comparison for futures-referenced contracts.
8. Numeric-threshold exactness: “\$50/bbl” vs. “\$55/bbl” rejected.
9. Event-key disambiguation: “Game 4” vs. “Game 5”.
10. Category-consistency guard: oil market cannot match non-oil event.

This filter rejected **140 of 615 pairs (22.8%)**. The impact was large: at the first large-scale backtest (April 11, 2026), linear-regression P&L went from **−\$5.28 to +\$5.45** after adding these rules — a $+\$10.73$ swing purely from removing structurally-bad matches, with no model changes. We interpret this as direct empirical evidence that data quality is more important than model choice at this scale.

3.3 Feature Engineering

We engineer **59 features** organized into five groups. Table 1 summarizes.

Table 1: Feature taxonomy (59 features total).

Group	Count	Examples	Motivation
Raw aligned (per-platform OHLCV)	18	<code>kalshi_open</code> , <code>polymarket_volume</code> , <code>kalshi_close</code>	Native platform signal
Cross-platform basic	6	<code>spread</code> , <code>mid_price</code> , <code>dollar_volume_ratio</code>	Arbitrage-signal primitives
Rolling/momentum	9	<code>spread_momentum_6</code> , <code>spread_volatility_6</code> , <code>spread_zscore</code>	Short-memory dynamics
Classical microstructure (academic)	13	<code>amihud_illiquidity</code> , <code>corwin_schultz_spread</code> , <code>kyle_lambda</code> , <code>roll_spread</code> , <code>bekker_parkinson_vol</code>	Grounded in 1984–2012 literature
Prediction-market-specific	13	<code>favorite_longshot_bias</code> , <code>near_expiry_indicator</code> , <code>platform_age_delta</code>	Domain-specific (Burgi 2026)

The microstructure features are computed per-platform then differenced across platforms, producing `amihud_illiquidity_delta`, `kyle_lambda_delta`, etc. The rationale is that *relative* liquidity across platforms should predict which side converges — a Polymarket spread that is less informed (higher Amihud, higher Roll spread) should pull toward the more-liquid Kalshi price.

Empirically, the 13 classical-microstructure features are *neutral-to-slightly-positive* on the historical dataset. This is because the features rely on rolling windows of depth 6–12 bars, and the historical dataset averages only 47 bars/pair. As live trading accumulates more bars per pair, these features are expected to become informative — a hypothesis we explicitly test via the data-scaling experiment (§4.3).

4. Methodology

4.1 Four Model Tiers

We compare four tiers of increasing complexity, all trained from scratch on the same matched-pair dataset using the same features and the same target variable $\Delta s(t)$.

Tier 0 — Naive baselines (lower bound): - *Naive-closes*: predict spread always closes fully by resolution. - *Volume-higher-wins*: predict higher-volume platform is always correct.

Tier 1 — Regression baselines (the backbone): - *Linear Regression* (scikit-learn `LinearRegression`). - *XGBoost* (xgboost `XGBRegressor`), searched over $\text{depth} \in \{3, 5, 7, 9\} \times \text{learning-rate} \in \{0.01, 0.05, 0.1, 0.3\} \times \text{n_estimators} \in \{100, 300, 500\}$ (48 configurations).

Tier 2 — Time-series models: - *GRU* (PyTorch, 64 hidden units, 1 layer, 24-bar lookback, StandardScaler inputs, Adam optimizer, `lr=1e-3`, early stopping on validation loss). - *LSTM* (same architecture, LSTMCell replaces GRUCell). - We attempted *TFT* (Temporal Fusion Transformer via PyTorch Forecasting, `hidden_size=8`, `attention_head_size=1`, `dropout=0.3`, 3-quantile QuantileLoss, GroupNormalizer per-pair) with pre-specified small-data hyperparameters; it did not converge at $N=6,802$ rows (see §6.2.3 and §6.4).

Tier 3 — Reinforcement learning: - *PPO-Raw*: PPO agent acting directly on 59-dimensional feature vectors, 3-action space {buy-spread, sell-spread, hold}, custom gym environment with mark-to-market reward at each step. PPO implementation from `stable-baselines3`. - *PPO-Filtered*: same PPO agent, but a trained autoencoder (3-layer symmetric, bottleneck 8) pre-filters observations, flagging “anomalous” spreads by reconstruction error. PPO only trades when anomalies are detected.

All models use the same train/test split: **time-ordered 80/20**, preserving temporal causality. No shuffling. No look-ahead. Features are computed using only past information.

4.2 Evaluation Protocol

Five evaluation regimes are used — each provides an independent view.

(a) Single-split backtest. Train on the first 80% of data (chronological), test on the last 20%. Report RMSE, MAE, directional accuracy, simulated P&L at 2pp fees, win rate, and per-trade Sharpe.

(b) Walk-forward backtest (11 windows). Concatenate train + test for maximum time coverage (Jan 1 – Apr 1, 2026 of the historical dataset). Split into 12 equal-time windows; use an expanding-window protocol where window i is trained on all data from windows $\{0, \dots, i-1\}$ and tested on window i (window 0 has no training set, so we report 11 evaluation windows). Per-window metrics: RMSE, directional accuracy, P&L at 2 pp, win rate, and per-trade Sharpe. If the edge is stable, we expect positive P&L across windows; if it is improving with more training data, we expect per-trade Sharpe to trend upward.

(c) **Per-category breakdown.** Stratify test-set rows by contract category (oil, crypto, inflation, employment, Fed, GDP, politics-election, politics-policy, sports) using a deterministic rule derived from Kalshi tickers and Polymarket slugs. Report per-category P&L, win rate, and trade count for each model.

(d) **Data-scaling curve.** Train each Tier-1 and Tier-2 model on progressively larger slices of the data — 50, 100, 250, 500, 1000, 2000 bars/pair — and plot P&L vs. training size. This answers “does the simple-beats-complex conclusion hold across scales, or is it an artifact of small data?”

(e) **Live paper trading.** Deploy the best-performing trained model (depth-3 XGBoost ensemble with LR) on BU SCC with a 15-minute trading cycle. Auto-discover new pairs every 3 hours, auto-retrain every 6 hours. Record trades, exits, and P&L in `positions.db` and `trade_log.jsonl`.

4.3 Training and Hyperparameter Search

Tier-1 models complete training in under 15 seconds on a single CPU core. Tier-2 models take approximately 3 minutes per epoch on a single CPU, early-stopping after 10–15 epochs. Tier-3 models take 20–40 minutes for PPO convergence.

The XGBoost hyperparameter sweep was exhaustive: **48 configurations**, evaluated on the single-split backtest. Results are in Table 5 (§5.3); the best configuration is `depth=3, lr=0.01, n=100`. All 10 top configurations used `depth ∈ {3, 5}` — deeper trees *lost* P&L, evidence of over-fitting at this training-set size (6,802 rows).

Tier-2 models were not searched over hyperparameters due to compute budget; we used architecture defaults from the original PyTorch Forecasting examples and report the result transparently.

4.4 Live System Architecture

The paper-trading system runs on BU SCC (`scc1.bu.edu`) with three cron jobs:

1. **Trading cycle** every 15 minutes on the login node (~3 min CPU) — fetches live prices, generates predictions via the trained LR + XGBoost ensemble, applies a category-aware entry filter (commodity pairs use the base threshold, non-commodity require $3\times$ confidence) and a concordance filter (skip if LR and XGBoost disagree on sign), then executes paper trades against the current market mid.
2. **Market discovery** every 3 hours as a batch job via `qsub` (~10 min) — fetches fresh Kalshi and Polymarket universes, matches semantically, applies quality filters, evicts stale pairs via a tombstone system with a 7-day TTL that protects open positions.
3. **Model retraining** every 6 hours as a batch job via `qsub` (~30 min) — rebuilds the training set from accumulated live bars and retrains LR, XGBoost, GRU, LSTM. A “checkpoint system” triggers the scaling-curve experiment when at least 20 pairs cross each data threshold (50, 100, 250, 500, 1000 bars/pair).

GitHub Actions runs both discovery and trading as *redundant fallback* workflows on a separate schedule. If SCC goes down (it has scheduled maintenance twice per semester), GHA keeps the system alive. All state (pair mappings, positions, trade logs, model artifacts) is committed to Git and synchronized via rebase-retry push logic.

A critical bug discovered during deployment and documented in §4.5 was that three code paths disagreed on what a `live_NNNN` pair ID meant, causing 25 positions to track wrong markets. This was fixed by moving to **content-addressed pair IDs** (e.g., `kxwti26apr08t10799-0x43d5953d`) derived deterministically from the normalized Kalshi ticker and Polymarket token ID.

Ensemble choice is evidence-based, not cherry-picked. The live system uses an LR + XGBoost ensemble with equal weights and a strict concordance filter (full evaluation in §5.11). Formal evaluation across four ensemble variants (LR alone; LR+XGB; LR+LSTM; LR+XGB+LSTM) and an 11-point LR-weight sensitivity sweep (0.0 → 1.0 in 0.1 steps) confirms this design. Ensemble weight choice is immaterial:

	filtered	P&L	spans	only	4.68acrosstheentireweightrange(+199.54 to +204.22)
LR and XGB are functionally tied on this dataset					(+201.69 and \$+199.54 solo respectively).

The concordance filter — not the weighting scheme — is the primary risk control: it rejects 4.80% of potential trades for variant (b) and improves per-trade Sharpe from 0.437 (unfiltered) to 0.455 (filtered). The filter has a measurable cost, however — rejected trades would have netted \$+1.95 in aggregate P&L had they been taken (P4 concordance-trap flag). We therefore report both filtered and unfiltered P&L in §5 to prevent Sharpe inflation. The formal `EnsemblePredictor` class (Phase 13, 13 passing unit tests) is not wired into the live strategy during v1.1 evaluation; the live system remains hardcoded to the current LR+XGB average for safety, with `src/live/strategy.py` untouched throughout Phase 13 (ENSM-05 guard).

4.5 Challenges Overcome

Three non-obvious infrastructure bugs materially affected our results:

1. **Kalshi /events silent HTTP 429.** The endpoint returned 429 Too Many Requests on roughly 40% of calls, *silently dropping entire commodity series*. Fixed with exponential backoff (1s/2s/4s/8s) and a per-series 250 ms pace. Commodity pair count went from 65 → **506** after the fix.
2. **Polymarket pagination too shallow.** We were fetching only the top 5,000 markets; WTI markets sat at offset 15,305+ and were completely invisible. Fixed by bumping `max_pages` from 10 to 60 (up to 30,000 markets). The complete commodity universe then became reachable.

3. **Gamma condition_id vs. condition_ids.** As noted above, the singular form returned *random unrelated markets*, not an error. We only discovered this when Polymarket prices came back for a Canadian-recession contract labeled as being about a Russia–Ukraine ceasefire. The fix (`condition_ids= plural`) was a one-character change, but the failure mode (silent, non-obvious) lost us several days of debugging.

These three bugs shared a common pattern: APIs that fail silently in ways that *look like* model problems. We interpret this as a general lesson — for ML systems on external APIs, infrastructure monitoring must come before model tuning.

5. Results

5.1 Headline Model Comparison

Table 2 shows the single-split backtest with a 2 pp signal threshold for trade entry on the full 1,673-row test set; transaction-cost sensitivity is analyzed separately in §5.6. (The Phase 18 audit confirmed `simulate_profit` charges zero fee — the `threshold=0.02` parameter is a SIGNAL gate for trade entry, not a transaction cost; see `AUDIT_REPORT.md` Tier 3.) All models use the same feature set (51 numeric features; 8 NaN/zero-variance columns excluded from the 59 engineered) and the same target variable (next-bar spread change). Results are sourced verbatim from `experiments/results/canonical/headline.json` (Phase 17-01 canonical regenerator under `seed=42`, `threshold=0.02`, `position_size=$100`). The headline numbers reproduce end-to-end via `python3 experiments/run_canonical.py`.

Table 2: Single-split backtest results (canonical, 2 pp signal threshold, 1,673-row test set). Per-trade Sharpe and per-trade alpha (in basis points at \$100 position size) lead the comparison; cumulative dollar P&L follows. PPO+autoencoder figure is the canonical \$+4.61 / 899-trade result (Phase 17-01 single-source-of-truth); the legacy ~\$−88K dollar-notional figure (`WalkForwardBacktester` units mismatch, ~200× contract scaling) is archived under `experiments/results/archive/` (see §6.3 and 17-02-PPO-DIAGNOSTIC.md).

Tier	Model	RMSE	Dir. Acc.	Sharpe (per-trade)	Alpha (bps/trade)	P&L (\$100 pos)	Win Rate	# trades
0	Naive (closes)	0.499	47.7%	0.125	+4.0	+\$58.12	47.9%	1460
0	Volume (higher wins)	0.457	47.7%	0.131	+4.2	+\$59.81	48.1%	1440
1	Linear Regression	0.306	56.9%	0.501	+15.0	+\$232.67	57.8%	1549
1	XGBoost (depth 3)	0.290	56.6%	0.499	+14.9	+\$232.83	57.4%	1559
2	LSTM	0.291	65.5%	0.473	+14.3	+\$221.84	56.5%	1547
2	GRU	0.293	64.3%	0.459	+14.0	+\$212.50	55.8%	1517
2	TFT†	0.326	51.7%	0.155	+5.5	+\$6.57	37.5%	120
3	PPO-Raw	0.319	60.5%	0.306	+9.6	+\$158.15	52.2%	1656
3	PPO + Autoencoder	0.327	27.2%	0.014	+0.5	+\$4.61	43.2%	899

†TFT: did not converge at N=6,802 (30-epoch budget, `hidden_size=8`, avg RMSE=0.3262 vs GRU=0.2928). VSN encoder weights extracted and visualized in Fig. 10 (see `experiments/figures/tft_variable_importance.png`). See §6.2.3 for full analysis.

Three observations. **First**, the ranking is unambiguous: **Tier 1 (regression) > Tier 2 (sequence) > Tier 3 (RL)**. The Tier-0 → Tier-1 gap is very large ($\approx 3.9\times$ in dollars, $\approx 4\times$ in per-trade alpha: 4 bps → 15 bps); the Tier-1 → Tier-2 gap is smaller but consistent ($\approx 5\text{--}9\%$ in P&L, $\approx 0.7\text{--}1.0$ bps in alpha); PPO+autoencoder collapses to essentially zero edge (+0.5 bps over 899 trades). **Second**, Linear Regression and XGBoost are *essentially tied* (Sharpe 0.501 vs 0.499; +15.0 bps vs +14.9 bps; \$+232.67 vs \$+232.83). LR wins per-trade Sharpe, alpha-per-trade, directional accuracy, and win rate; XGBoost wins RMSE and total P&L by a hair. Within Tier 1, there is no statistically defensible reason to prefer one over the other on this dataset. **Third**, LSTM edges GRU by 0.3 bps per trade and \$9 in P&L in this single split, reversing the walk-forward median ordering (§5.2); both sit $\approx 0.7\text{--}1.0$ bps below Tier 1.

Two supplementary figures accompany this headline split. Fig. 6 (`experiments/figures/backtest_equity_curves.png`) plots cumulative test-set P&L bar-by-bar for the four ML models and shows the Tier-1 vs. Tier-2 separation opening gradually rather than in a single jump — consistent with a stable per-trade edge rather than one lucky window. Fig. 7 (`experiments/figures/bootstrap_ci_rmse.png`) reports bootstrap 95% confidence intervals on RMSE (1,000 resamples); the LR / XGBoost / GRU / LSTM intervals overlap heavily on RMSE even though their P&L separates cleanly, confirming that the P&L gap is driven by *directional accuracy* and *trade selection*, not raw regression error.

5.2 Walk-Forward Validation

Fig. 1 (see `experiments/figures/walk_forward_pnl.png`) shows per-window P&L for all six models across 11 windows spanning January to April 2026. Training uses an expanding-window protocol: window i trains on all data from windows $\{0, \dots, i-1\}$ and tests on window i . Table 3 reports per-window P&L; Table 3b reports per-trade Sharpe across the same windows (the key stability statistic).

Table 3: Walk-forward P&L at 2 pp fees (expanding-window, 11 out-of-sample windows).

Window	Train rows	Test rows	Naive	Volume	LR	XGBoost	GRU	LSTM
1	357	558	+\$34.57	+\$31.95	+\$48.70	+\$53.50	+\$53.08	+\$41.93
2	915	648	+\$29.91	+\$28.76	+\$61.33	+\$58.70	+\$59.92	+\$55.52
3	1,563	963	+\$62.50	+\$65.60	+\$106.78	+\$109.25	+\$104.89	+\$93.68
4	2,526	1,318	+\$40.14	+\$40.10	+\$141.50	+\$142.44	+\$134.71	+\$135.00
5	3,844	1,125	+\$21.25	+\$24.39	+\$128.85	+\$125.98	+\$120.44	+\$120.83
6	4,969	680	+\$10.64	+\$9.08	+\$80.42	+\$76.65	+\$77.81	+\$73.72
7	5,649	651	+\$25.77	+\$22.84	+\$65.70	+\$67.36	+\$64.82	+\$65.32
8	6,300	1,123	+\$25.71	+\$26.87	+\$144.89	+\$138.68	+\$135.85	+\$141.08
9	7,423	589	+\$14.67	+\$13.92	+\$73.38	+\$74.29	+\$70.51	+\$75.18
10	8,012	497	+\$15.53	+\$16.61	+\$75.30	+\$75.27	+\$75.28	+\$73.91
11	8,509	110	+\$2.72	+\$2.87	+\$10.22	+\$11.54	+\$9.34	+\$10.67
Median			+\$25.71	+\$24.39	+\$75.30	+\$75.27	+\$75.28	+\$73.91
Mean			+\$25.76	+\$25.73	+\$85.19	+\$84.88	+\$82.42	+\$80.62

Table 3b: Walk-forward per-trade Sharpe ratio by window.

Window	LR	XGBoost	GRU	LSTM
1	0.307	0.383	0.341	0.260
2	0.355	0.347	0.367	0.321
3	0.408	0.424	0.410	0.359
4	0.403	0.403	0.383	0.386
5	0.448	0.445	0.425	0.417
6	0.453	0.424	0.441	0.411
7	0.405	0.426	0.405	0.405
8	0.493	0.464	0.449	0.476
9	0.440	0.445	0.435	0.448
10	0.530	0.540	0.537	0.529
11	0.297	0.350	0.272	0.328
Median	0.408	0.424	0.410	0.405

Four findings emerge:

1. **Every window is positive for every ML model.** 11/11 out-of-sample windows, 4/4 ML models — zero losing windows. This is strong evidence the edge is not a lucky train/test split.
2. **Per-trade Sharpe trends upward across windows.** LR rises from 0.307 in window 1 (357 training rows) to 0.530 in window 10 (8,012 training rows) — a **73% improvement** as training data accumulates. XGBoost shows the same pattern (0.383 → 0.540). Window 11 drops because its test set has only 110 rows and is statistically noisy. The trajectory is consistent with classic time-series ML: more data → tighter risk-adjusted returns.
3. **All four ML models converge to near-identical median performance** (\$73.91–\$75.30 median P&L). The walk-forward does not cleanly separate XGBoost from LR, GRU, or LSTM. This is the strongest indication that at current data scale **model family essentially does not matter** among reasonable Tier 1 + Tier 2 choices — what matters is whether any ML model is used at all.
4. **The ML vs. naive gap is ~3×** (median \$75 vs \$25–\$26), dwarfing the <\$2 differences between ML models. The economically meaningful comparison is *any ML model* vs. *no ML*, not *XGBoost* vs. *GRU*.

5.3 Per-Category Breakdown

Table 4 shows the single-split P&L stratified by contract category.

Table 4: Per-category model performance (single-split test set).

Category	# trades	LR P&L	XGBoost P&L	Winner	Win-Rate (LR)
Inflation	616	+\$89.39	+\$89.38	LR (tied)	63%
Crypto	292	+\$41.75	+\$48.14	XGBoost	57%
Politics–policy	278	+\$29.76	+\$31.03	XGBoost	32%
Employment	204	+\$20.02	+\$19.94	LR (tied)	47%
Politics–election	129	+\$17.95	+\$17.55	LR	28%
GDP	20	+\$0.91	+\$0.91	tie	15%
Fed rates	10	+\$1.90	+\$1.90	tie	50%
Overall	1,549	+\$201.69	+\$208.85	XGBoost	51%

Linear regression wins 5 of 7 categories with sufficient trade count, but XGBoost wins crypto by a notable \$6 margin and politics-policy by \$1.30. *The apparent “XGBoost wins overall” result is driven almost entirely by crypto outperformance* — consistent with XGBoost’s tree-based splits capturing crypto’s nonlinear volatility regimes that LR cannot. This is a nuanced finding: **the complexity premium is regime-specific, not universal.**

Additionally, on the live dataset (April 11 snapshot, 1,881 historical trades), oil near-expiry contracts stand out dramatically:

- **Oil:** 765 trades, **76.5% win rate**, **+\$0.41/trade** — a +142.7% edge over the pooled model.
- Fed rates: 431 trades, 34.6% WR, +\$0.01/trade.
- Sports: 618 trades, 37.4% WR, −\$0.00/trade.
- Politics: 67 trades, 29.9% WR, −\$0.02/trade.

Oil-contract convergence is largely mechanical: WTI-futures expiry dates resolve on observable market prices within hours of contract settlement, so cross-platform prices must converge. Sports and politics resolve on discrete events with no convergence dynamics — hence the poor performance. **The alpha is in the asset class as much as in the model.**

5.4 Data-Scaling Curve

Fig. 2 (see `experiments/results/data_scaling/pnl_at_2pp_vs_data.png`) shows the 6-point scaling curve. Plateau occurs because `train.parquet` contains at most 141 bars/pair (N=6,802 rows, 144 pairs); slices at 250+ bars/pair are identical to the 100-bar slice and produce identical metrics. Table 5 summarizes.

Table 5: Data-scaling experiment (P&L at 2pp).

Bars/pair	Training rows	LR P&L	XGBoost P&L	GRU P&L	LSTM P&L
50	4,646	+\$202.93	+\$210.57	—	—
100	6,290	+\$200.36	+\$211.07	+\$186.67	+\$182.76
250	6,802	+\$199.90	+\$210.01	+\$196.40	+\$181.85
500	6,802	+\$199.90	+\$210.01	—	—
1000	6,802	+\$199.90	+\$210.01	—	—
2000	6,802	+\$199.90	+\$210.01	—	—

Footnote: Rows 50 and 100 bars/pair used the 29-feature pipeline (April 11, 2026 batch run). Row 250 bars/pair uses the 29-feature pipeline (April 22, 2026 manual run, Phase 8 aligned). Both pipelines show the same qualitative ranking. Rows 500, 1000, 2000 are plateau-equivalent to row 250 (training data is capped at 6,802 rows, max 141 bars/pair); GRU/LSTM not re-run for those rows as they produce identical training slices.

The scaling curve plateaus at 100 bars/pair because `train.parquet` is capped at 6,802 rows and 144 pairs (max 141 bars/pair); slices beyond 100 bars/pair are identical to the 100-bar slice. Within that range, **the ranking is invariant:** XGBoost > LR > GRU > LSTM. This ranking holds at all three measured scale points (50, 100, 250 bars/pair), confirming invariance across a 5× growth in training data. This directly refutes the hypothesis that sequence models would overtake regression if only they had more data, at least for sample sizes reachable on our dataset.

5.5 XGBoost Hyperparameter Sweep

Table 6 shows the top 10 of 48 XGBoost configurations. All top-10 configurations use tree depth 3–7; depth 9 does not appear until later. The best configuration — **depth 3, learning rate 0.01, 100 trees** — is essentially “an ensemble of decision stumps,” confirming the “simple wins” theme even within a single model family.

Table 6: Top 10 XGBoost configurations (48 total).

Rank	depth	lr	n_est	RMSE	DA	P&L	Win%
1	3	0.01	100	0.288	57.7%	+\$209.70	58.1%
2	3	0.01	500	0.282	58.1%	+\$208.62	58.5%
3	5	0.01	500	0.285	57.9%	+\$208.29	58.4%
4	3	0.05	500	0.292	58.0%	+\$207.72	58.4%
5	3	0.05	100	0.282	58.1%	+\$207.69	58.3%
6	3	0.01	300	0.281	57.7%	+\$207.62	58.1%
7	5	0.01	100	0.285	57.8%	+\$207.38	58.2%
8	5	0.01	300	0.282	57.3%	+\$207.07	58.3%
9	7	0.01	100	0.284	57.3%	+\$206.83	58.0%
10	5	0.05	100	0.285	57.7%	+\$206.70	58.0%

5.6 Transaction-Cost Sensitivity

Table 7: Sensitivity to transaction costs (XGBoost, depth-3, fresh run April 17, 2026).

Fee (pp)	P&L	Win Rate	Sharpe/trade	# trades
0 (gross)	+\$245.06	57.6%	0.502	1,673
2 (our simulation)	+\$208.85	50.9%	0.449	1,567
3 (maker + small slippage)	+\$194.03	48.8%	0.452	1,410
5 (Kalshi taker maximum)	+\$165.53	46.1%	0.438	1,201
7 (adversarial worst-case)	+\$135.34	42.9%	0.407	1,018

The system is **profitable at every fee level** tested, from 0 pp (gross) up to the worst-case 7 pp adversarial assumption. Net P&L compresses roughly linearly with fees (−\$18 P&L per 1 pp of fee increase), and Sharpe is remarkably flat — in fact *rising slightly* from fee 2 to fee 3 because the higher fee filters out lower-confidence trades. In practice, Kalshi **maker** orders pay \$0 fee and Polymarket charges only ~1 pp in Polygon gas, so our 2 pp simulation is *conservative*.

Note that win rate decreases with fees — trades that just barely won at 0 pp lose at higher fees — but per-trade Sharpe is preserved because the lost trades had low magnitude. This is the statistical fingerprint of a *real* signal rather than noise: the signal survives cost scrubbing.

5.7 SHAP Feature Importance

SHAP analysis on the trained XGBoost model (see `experiments/figures/shap_bar_plot.png`) shows `polymarket_vwap` dominating feature importance with a mean $|\text{SHAP}|$ of ≈ 0.14 — twice the next feature. This suggests **Polymarket may be the “less efficient” side**, with its prices carrying more predictive information about future spread direction than Kalshi’s do. Interpreted loosely: Kalshi prices move first, Polymarket prices catch up — which is consistent with Kalshi’s smaller user base having a more concentrated informed flow and Polymarket’s larger retail user base producing a slower reaction function.

5.8 Honest Sharpe-Ratio Accounting

Per-trade Sharpe is the load-bearing headline. It is invariant to the leakage correction, sample-size-stable, and reproducible from canonical code. The per-pair annualized Sharpe is a regime-dependent secondary statistic — sensitive to the cross-pair correlation structure of the test pairs, which itself is sensitive to sample size and split methodology.

The Phase 18 audit reproduced these numbers from the raw trade ledger via `experiments/audit/audit_sharpe.py` (canonical) and `experiments/audit/audit_sharpe_purged.py` (leakage-free). Per-trade Sharpe drifts only +2.99% between the leaky and purged splits — well within the bootstrap CI. Per-pair Sharpe moves more (the Bailey-López de Prado (2012) effective-sample correction $n_{\text{eff}} = N / (1 + (N - 1) \times \text{avg_corr})$ is the active ingredient compressing the leaky number, and it does not apply to the purged sample because the purged cross-pair correlation is negative and the correction short-circuits when $\text{avg_corr} \leq 0$). See `AUDIT_REPORT.md` Tier 1 for the full assumption stack.

Table 8: Sharpe-ratio accounting (Linear Regression, Phase 18 audit reproduction). The leaky column reproduces the canonical 80/20 row-index split numbers; the purged column reports the leakage-free pair-stratified rebuild (115 train pairs / 29 test pairs, seed=42).

Method	Leaky canonical	Purged (leakage-free)	Interpretation
Per-trade (load-bearing)	0.501	0.516	Drift +2.99%; within bootstrap CI; the headline.
Per-pair (naive)	0.781	0.814	Each pair = one independent bet; raw cross-pair Sharpe.
Per-pair (BLdP-corrected)	0.296	0.814	Leaky avg pairwise corr +0.042 → BLdP haircut active; purged avg pairwise corr −0.199 → BLdP short-circuited (<code>avg_corr ≤ 0</code> returns naive).
Per-pair 95% CI	[0.685, 0.904]	[0.700, 1.067]	10,000 bootstrap resamples.
Per-pair $\times \sqrt{(\text{pairs}/\text{year})}$, naive	≈ 18.6	≈ 8.93	Annualization factor $\sqrt{(\text{pairs_per_year})}$: leaky 23.8, purged 11.0. Sample-size-sensitive; reported for completeness only.
Per-trade alpha (bps)	+15.0	+15.7	Drift +4.5%; same trade-level economics.

The honest framing: the per-trade edge of ≈ 0.50 Sharpe / +15 bps alpha is the load-bearing claim — it survives the embargo audit (Tier 2) and reproduces under both splits. The per-pair annualized number (range bracketed by 0.30 leaky-corrected and 0.81 purged) is regime-dependent and reported with full caveats: (a) annualization assumes pair-lifecycle distribution in the test window is representative of annual operation (likely violated; flagged in §6.4); (b) the BLdP correction's applicability flips between sample regimes; (c) at $N=29$ purged test pairs the cross-pair correlation matrix is sample-size-sensitive. We report this transparently.

Industry context: Sharpe 0.5 per-trade is in line with mid-frequency stat-arb strategies (1–5 bps $\times$ thousands of trades). Sharpe 1.0 (per-pair, naive) is plausible for a strategy with structural edge; Sharpe 2–3 (annualized, naive) is the upper bound under favorable annualization assumptions. The earlier draft cited a per-pair annualized Sharpe of ≈ 3.2 in the abstract; the audit found this number had no derivation path in the codebase. The leakage-free recompute replaces it with the per-trade-led headline above.

Per-trade alpha in basis points. Per-trade Sharpe 0.501 (leaky) → 0.516 (purged) is equivalent to **+15.0 → +15.7 bps of alpha per trade at \$100 position size** (`alpha_bps_per_trade = total_pnl / num_trades / position_size × 10,000`; canonical formula in `experiments/run_canonical.py`). For context, professional statistical-arbitrage strategies typically target 1–5 bps/trade with much higher trade counts (millions vs. our 1,549). Our 15 bps/trade reflects the larger-than-typical mispricing in immature cross-platform prediction markets — the alpha is real but the trade-count ceiling caps total scalability. PPO+autoencoder at +0.5 bps/trade essentially has no edge; PPO-Raw at +9.6 bps is mildly profitable but ~5 bps below the regression baselines. This per-trade-alpha framing is the pitch-standard quant headline; we report it alongside Sharpe so the system's edge is comparable to other fixed-position-size momentum / stat-arb strategies without unit conversion.

5.9 Live vs Backtest Reconciliation

The live paper-trading system was deployed on the BU Shared Computing Cluster (SCC) on April 11, 2026, running a 15-minute trade cycle and retraining models every six hours. It accumulated closed positions that can be independently verified by replaying each position's entry bar through the same deployed models (Linear Regression and XGBoost, the Tier 1 ensemble used in production). This shadow simulation constitutes a true out-of-sample live deployment test: the models were not updated between position entry and the reconciliation run, so there is no look-ahead contamination.

Data window and coverage. The live positions database (`data/live/positions.db`) contains 10,154 closed positions with entry timestamps spanning April 14–22, 2026 (eight days of live operation after the system stabilized post-deployment), accumulating 145,136 bars across 8,421 pairs. All 10,154 positions were successfully matched to an entry bar in `data/live/bars.parquet`, yielding a 100% match rate. The acceptance gate threshold of 80% was exceeded with no margin needed (gap metric: 0.00%).

Metric	Value
Total positions in window	10,154
Matched to entry bar	10,154 (100.0%)
Unmatched	0
Acceptance gate	PASSED (threshold: $\geq 80\%$)

Fee model disclaimer. Shadow-simulation P&L uses the threshold-only fee model consistent with `profit_sim.simulate_profit`: a trade is taken when $|\text{prediction}| > 2$ pp, and P&L equals the actual spread change times the sign of the prediction, with no explicit cost deducted. Table 2 P&L uses a 2 pp transaction-cost deduction model (`verify_headline.simulate_pnl`): winning trades subtract 2 pp from gross P&L, and losing trades add 2 pp to gross loss. The two are not directly comparable in absolute terms; this reconciliation focuses on directional accuracy and tracking error.

Summary comparison.

Metric	Value
Live realized P&L	+\$1.53
Shadow-simulation P&L	−\$1.53
Tracking error (live − sim)	+\$3.06
Gap metric (unmatched / total)	0.00%

Category breakdown.

Category	Count	Live P&L	Sim P&L	Tracking Error
other	6,005	+\$22.04	−\$22.04	+\$44.08
inflation	2,654	+\$0.23	−\$0.23	+\$0.46
crypto	915	−\$19.87	+\$19.87	−\$39.74
gdp	459	−\$0.65	+\$0.65	−\$1.30
politics_policy	71	−\$0.20	+\$0.20	−\$0.40
fed_rates	50	−\$0.01	+\$0.01	−\$0.02

Exit-reason attribution.

Exit Reason	Count	Live P&L	Sim P&L	Tracking Error
TAKE_PROFIT	88	+\$32.60	−\$32.60	+\$65.20
TIME_STOP	3,819	+\$11.83	−\$11.83	+\$23.66
RESOLUTION_EXIT	5,502	−\$9.90	+\$9.90	−\$19.80
MOMENTUM	634	−\$5.70	+\$5.70	−\$11.40
STOP_LOSS	111	−\$27.31	+\$27.31	−\$54.62

Findings.

- **Systematic directional anti-correlation persists at 4× the data.** The shadow simulation produces exactly the inverse P&L of the live system (+\$1.53 vs −\$1.53), with a tracking error of \$3.06. This exact anti-correlation now holds across 10,154 trades and eight days of operation, confirming it is not statistical noise. It is a structural consequence of model semantics: the deployed regression models predict the *next-bar spread change* (a mean-reversion signal: large positive spread → predict positive Δ spread). The live strategy enters *short_spread* on large spreads (betting the spread closes). Shadow simulation uses `sign(prediction)` as trade direction and therefore takes a long-spread position, exactly opposing the live entry. Notably, the per-trade tracking error has *decreased* relative to the April 16 snapshot (\$12.06 on 2,530 trades vs \$3.06 on 10,154 trades), indicating the anti-correlation is stable and does not amplify with volume. The strategy’s edge comes from spread-magnitude thresholding, not from directional alignment with the model predictions.
- **Positive-skew pattern dramatically confirmed.** TAKE_PROFIT triggered 88 times (0.87% of all trades) and generated +\$32.60 — more than 21× the net live P&L of +\$1.53. This is the classic asymmetric payoff structure documented in Finding 14: many small scratches absorbed by STOP_LOSS (111 trades, −\$27.31) and MOMENTUM exits (634 trades, −\$5.70), offset by infrequent but large TAKE_PROFIT events. At the April 16 snapshot, only 1 TAKE_PROFIT event had been observed; the 8-day data now provides a statistically meaningful characterization of this tail.
- **Crypto regime flip: non-stationarity caveat.** At the April 16 snapshot, crypto was the best-performing category (+\$4.33 across 261 trades). At the April 22 8-day snapshot, crypto has become the worst-performing category with real volume (−\$19.87 across 915 trades). This regime flip illustrates a genuine risk for per-category claims in the paper: category-level P&L can reverse sign over days, not months. Any strategy that calibrated on the 3-day crypto performance would have been systematically wrong at 8 days. We report this as a stationarity limitation.
- **“Other” category dominates volume and has tagging limitations.** The “other” bucket contains 6,005 trades (59% of total) and generates the largest gross live P&L contribution (+\$22.04). Investigation of the top ticker prefixes reveals that KXPAYROLLS (525 trades, employment) and KXEZCPIYOYF (86 trades, EZ CPI inflation) are being misclassified by the `derive_category_from_ticker` function. These should be tagged as `employment` and `inflation` respectively. This is a `category.py` limitation that does not affect the reconciliation’s directional conclusions — the anti-correlation finding is ticker-agnostic — but it means per-category P&L breakdowns understate inflation and do not capture any employment-category signal. We flag this as future work.

Finding: WTI oil contracts absent. WTI oil contracts were not present even in the expanded April 14–22 live trading window (zero closed positions). The commodity discovery gap (Kalshi 429 rate-limiting + Polymarket shallow pagination) was patched on April 11, but WTI contracts discovered after that date either have not yet entered and exited positions within this window or have insufficient price divergence

to cross the entry threshold. Finding 6 (oil near-expiry edge, \$5.3) cannot be independently tested on live data; this is explicitly acknowledged as a limitation. The oil edge finding (76.5% win rate, +\$0.41/trade, +142.7% P&L vs pooled) remains a backtest finding and cannot be directly validated on live data within this study's time window.

Paper-trading caveats. No slippage is modeled: trades execute at mid-price with zero market impact. No partial fills: all trades are assumed fully filled at the stated price. No margin: capital is notional (the system does not track margin consumption or liquidation risk). These idealizations mean live P&L represents an upper bound on what a real-money implementation would achieve. In practice, Kalshi maker orders pay \$0 fee and Polymarket charges approximately 1 pp in Polygon gas, consistent with our 2 pp simulation assumption being conservative.

5.9.1 Post-Submission Commodity-Matching Fix (Phase 15, April 24)

The §5.9 reconciliation window above (April 14–22) closed with zero WTI oil positions, which we reported as an unresolved engineering limitation (§6.4 item 9). Between the April 22 snapshot and submission, we diagnosed and patched the root cause: `KALSHI_DISCOVERY_CATEGORIES` in `src/live/market_discovery.py` omitted the string "Commodities", and Kalshi had migrated daily WTI / Brent / grain / metal series into that category after an internal taxonomy change. The series were never reaching the discovery sweep. A single-line tuple edit (commit `38d7970`), combined with a classifier extension in `src/features/category.py` for the Brent family plus daily-WTI variants, and a 200-slot commodity reservation inside `src/live/collector.py::load_live_pairs` to prevent similarity-cap eviction (data regen commit `d217ff1`), unblocked the pipeline end-to-end.

Post-fix live validation (12-hour window, April 24). After deployment on the BU SCC, a 12-hour observation window (2026-04-24T01:28Z through 2026-04-24T13:00Z) closed **1,224 non-KXWTIMAX commodity positions**, exceeding the pre-registered validation target (≥ 10 closed positions) by 122×. Aggregate paper-trading P&L over the window was **+\$1.96**; win rate was **36.0%** (441 / 1,224). Per-series breakdown:

Kalshi series	Closed positions	Description
KXBRENTW	486	Weekly Brent range
KXWTI	409	Daily WTI on-day
KXWTIW	213	Weekly WTI range
KXBRENTMON	76	Monthly Brent
KXBRENTD	16	Daily Brent
KXAAAGASD	11	Daily AAA retail gasoline
KXAAAGASW	7	Weekly AAA retail gasoline
KXAAAGASM	6	Monthly AAA retail gasoline
Total	1,224	

Interpretation caveats. This 12-hour window is a *proof-of-life* result, not a robust live edge measurement. Three honest caveats bound what can be claimed: (a) the window is less than half a full trading day and spans one market regime; (b) the +\$1.96 aggregate P&L is dollar-positive but economically near-flat across 1,224 positions ($\approx$ \$0.0016 per trade), dramatically lower than the backtest oil near-expiry edge (Finding 6, \$5.3: +\$0.41/trade at 76.5% win rate) — the live window includes *full-contract-lifecycle* positions, not only the near-expiry subset the backtest measured, so a lower win rate is expected rather than contradictory; (c) the paper-trading caveats from §5.9 above (no slippage, no partial fills, no margin) apply unchanged. What this result *does* establish: the discovery / matching / collector / trading-cycle stack now trades daily and weekly WTI and Brent end-to-end, which was not true at the April 22 snapshot. Future work can now measure live oil-edge stability over longer windows on this same code path.

5.10 Feature Ablation

To guard against p-hacking, we pre-registered the ablation protocol at `.planning/ablation_protocol.md` before any experiment script was written (the protocol commit `b15534b` predates the runner commit `46b253a`, verifiable in the repository history). We applied Leave-One-Group-Out (LOGO) ablation across five pre-specified feature groups on both LR and XGBoost, using a three-way temporal split: `train_proper` (earliest 85% of training data, 5,781 rows), `ablation_holdout` (latest 15%, 1,021 rows) for feature-set selection, and `final_test` (1,673 rows, the original held-out test set) frozen until after selection was determined.

Feature Groups. The 51 model features were partitioned into five non-overlapping, pre-specified groups: (A) raw aligned OHLCV (15 features), (B) cross-platform basics including spread and divergence metrics (10 features), (C) rolling and momentum indicators (6 features), (D) classical microstructure estimators — Amihud illiquidity, Kyle's lambda, Roll spread, Corwin–Schultz implied spread, high-low volatility (13 features), and (E) prediction-market-specific dynamics including longshot bias score and trade-size extremes (7 features).

Results. Table 9 presents all 12 LOGO configurations (6 per model). No configurations were omitted.

Table 9. LOGO feature-ablation results on `ablation_holdout` (1,021 rows). P&L evaluated at 2 pp fee threshold. Δ P&L and 95% CI computed via 1,000 paired-bootstrap resamples of trade indices. All CIs computed on `ablation_holdout` only; `final_test` is frozen.

Model	Dropped Group	# Features	P&L @ 2pp	Δ P&L	95% CI of Δ P&L	RMSE	Dir. Acc.	Classification
LR	— (baseline)	51	\$+56.54	\$0.00	—	0.2192	62.0%	baseline
XGBoost	— (baseline)	51	\$+54.00	\$0.00	—	0.2234	60.8%	baseline
LR	A — Raw OHLCV	36	\$+53.21	\$-3.33	[-6.80, -0.12]	0.2241	61.2%	droppable
LR	B — Cross-platform	41	\$+56.72	\$+0.18	[-0.51, +1.05]	0.2191	62.0%	droppable
LR	C — Rolling/momentum	45	\$+56.23	\$-0.31	[-2.06, +1.32]	0.2168	61.8%	droppable
LR	D — Microstructure	38	\$+56.54	\$-0.00	[-0.88, +0.72]	0.2187	61.9%	droppable
LR	E — Pred-market	44	\$+56.77	\$+0.23	[-0.30, +0.99]	0.2193	61.9%	droppable
XGBoost	A — Raw OHLCV	36	\$+52.32	\$-1.68	[-5.67, +2.40]	0.2248	61.4%	droppable
XGBoost	B — Cross-platform	41	\$+55.08	\$+1.08	[-1.48, +4.05]	0.2265	60.7%	droppable
XGBoost	C — Rolling/momentum	45	\$+53.59	\$-0.41	[-3.74, +2.99]	0.2242	61.2%	droppable
XGBoost	D — Microstructure	38	\$+53.43	\$-0.57	[-4.30, +2.66]	0.2295	60.7%	droppable
XGBoost	E — Pred-market	44	\$+52.85	\$-1.15	[-4.56, +2.01]	0.2316	61.1%	droppable

Statistical Power Limitation. At $N=1,021$ ablation-holdout rows, we could not detect statistically significant load-bearing effects for any feature group. All 10 drop-group configurations have 95% CIs whose absolute delta is less than \$10, and nine of the ten CIs straddle zero. The one exception — LR drop-A, 95% CI $[-6.80, -0.12]$ — is technically entirely below zero but has a mean delta of only $-\$3.33$, placing it well below the \$10 load-bearing threshold. This is not a finding that all feature groups are equivalent; it is an honest statement about statistical power. Paired-bootstrap CIs computed on 1,021 rows are wide by construction, and real group-level effects smaller than $\sim\$10$ are undetectable at this holdout size.

Minimum Sufficient Set Determination. Because no group meets the pre-registered load-bearing criteria (95% CI fully below zero AND $|\text{delta}| > \$10$ for both models), the ablation yields an inconclusive classification for all groups under the pre-specified protocol. We therefore conservatively retain all 51 features as the operational feature set. This decision preserves the full model from $\$5.1$ – $\$5.2$ without modification and avoids any post-hoc feature selection that could introduce look-ahead bias. The final_test set (1,673 rows) was not evaluated on any reduced-feature variant; the one-shot generalization metric for feature selection is deferred to a future ablation run with sufficient statistical power.

Future Work. Classical microstructure estimators (Group D: Amihud illiquidity, Kyle’s lambda, Roll spread, Corwin–Schultz spread) are Nyquist-starved at the current 4-hour bar interval and limited sample size; ablation should be re-run at 250+ bars/pair where these estimators become identifiable with less noise (see §7, item 8). At that scale, the ablation-holdout will contain substantially more rows and bootstrap CIs will be tight enough to classify individual groups as load-bearing or droppable with high confidence.

5.11 Ensemble Formalization

The live trading system deploys an LR + XGBoost ensemble with a concordance filter: trades are skipped when the two models disagree on direction (sign disagreement). We formally evaluate four ensemble variants and audit the concordance filter’s effect on reported performance. All numbers below are sourced verbatim from `experiments/results/ensemble/summary.json` produced by `experiments/run_ensemble_sweep.py`.

Table 10. Ensemble-variant comparison with concordance audit on the held-out test set (1,673 rows). P&L is reported at the 2 pp fee threshold, both with the concordance filter applied (“filtered”) and with the filter removed (“unfiltered”). Rejected P&L is the counterfactual cumulative P&L that the concordance filter *discarded* — i.e., what the rejected trades would have earned had they been taken. The P4 flag fires when rejected P&L is net positive (the filter throws away profitable trades).

Variant	Members	Concordance	Trades (filtered)	Trades (unfiltered)	P&L (filtered)	P&L (unfiltered)	Rejection Rate	P&L (rejected)	P4 flag
(a) LR alone	LR	none	1549	1549	+\$201.69	+\$201.69	0.00%	+\$0.00	—
(b) LR + XGB equal-weight	LR, XGB	strict	1489	1564	+\$202.14	+\$204.09	4.80%	+\$1.95	Yes
(c) LR + LSTM equal-weight	LR, LSTM	strict	1441	1550	+\$191.79	+\$201.31	7.03%	+\$9.52	Yes
(d) LR + XGB + LSTM strict	LR, XGB, LSTM	strict	1373	1554	+\$194.86	+\$207.93	11.65%	+\$13.08	Yes

Concordance filter (“strict” mode): a trade is taken only when all ensemble members agree on the sign of the predicted spread change. P&L (rejected) is the counterfactual P&L had rejected trades been taken. P4 flag fires when rejected trades are profitable in aggregate — i.e., the filter is discarding net-positive expected value.

Weight Sensitivity. To test whether the equal-weight choice is cherry-picked, we sweep LR-weight from 0.0 to 1.0 in 0.1 increments for the LR + XGB variant, holding XGB-weight = 1 − LR-weight, and re-evaluate both filtered and unfiltered P&L at each point. A fresh `EnsemblePredictor` is fit at each weight step (rather than in-place weight mutation) to guarantee train-time isolation, and `set_all_seeds(42)` is invoked before every fit.

Fig. 11 (`experiments/figures/ensemble_weight_sweep.png`) plots walk-forward P&L at 2 pp fees across the 11-point LR-weight sweep (0.0 → 1.0). The spread across weights is \$4.68; the curve is essentially flat, confirming that weight choice is not cherry-picked.

The weight sweep is near-flat across all 11 tested values. Filtered P&L spans only \$4.68 (from \$+199.54 at w=0.0 to \$+204.22 at w=1.0); unfiltered P&L spans \$6.30 (from \$+201.63 at w=0.0 to \$+207.21 at w=0.2). This is consistent with the Phase-13 prior expectation: LR and XGBoost are functionally tied on this dataset (LR solo P&L = \$+201.69, XGB solo P&L — the w=0.0 point of the filtered sweep — = \$+199.54), so convex combinations of them produce only second-order variation. The LR-XGB weight choice is not material.

The concordance filter is the real discriminator — and it has a measurable cost. Across all three filtered variants (b), (c), and (d), the concordance filter fires the P4 warning: rejected trades were profitable in aggregate. For the live ensemble (variant b), the filter rejected 4.80% of potential trades with aggregate rejected P&L of \$+1.95. The more complex ensembles reject more and discard more: variant (c) LR+LSTM rejects 7.03% with \$+9.52 rejected P&L; variant (d) LR+XGB+LSTM rejects 11.65% with **\$+13.08 rejected P&L. For variant (d), the filter's cost is particularly visible — filtered P&L (+194.86) is 13.07 *lower* than unfiltered P&L (+207.93).** The filter is trading real expected P&L for variance reduction (higher per-trade Sharpe: 0.475 filtered vs. 0.452 unfiltered for variant d).

This is the P4 concordance-filter denominator trap identified in the Phase-13 research design: the filter improves reported Sharpe by selectively eliminating ambiguous trades, *some of which have positive expected value*. We report both filtered and unfiltered P&L throughout to prevent Sharpe inflation, and flag rejected-profitable cases explicitly in `summary.json` via the `flag_rejected_profitable` field.

Interpretation. The live ensemble (variant b, LR + XGB equal-weight, strict concordance) is evidence-justified under an honest reading of the evidence: the ensemble’s weight choice is immaterial (sweep span \$4.68), and the concordance filter is the primary risk control. But the filter has a quantified cost — 4.80% of trades are rejected, and rejected trades would have netted \$+1.95 had they been taken. The filtered P&L of +202.14 is *modestly higher* than LR solo (+201.69) but *lower* than unfiltered LR+XGB (\$+204.09). On this dataset, the ensemble’s net contribution over LR-solo is **+0.45 P&L and a small Sharpe improvement (0.455 vs. 0.434 per-trade filtered).** The more aggressive LSTM-including variants (c) and (d) reject more, **co +191.79 vs. \$+201.69).** The simplest two-member ensemble is the right operational choice.

The `EnsemblePredictor` class is formalized and tested (`tests/models/test_ensemble.py` , 13 passing tests; Phase 13 Plan 01) but *not wired into the live strategy during v1.1 evaluation* — the live system remains hardcoded to the current LR+XGB average for safety. Wiring `EnsemblePredictor` into `src/live/strategy.py` is a post-v1.1 refactor (`ENSM-05` guard; see §7).

5.12 Lookback Window Sensitivity (Proposal Experiment 2)

The original project proposal (Experiment 2) asked whether the length of the historical lookback window materially affects sequence-model performance — i.e., does feeding GRU/LSTM a longer history of prior bars improve P&L? To answer this, we sweep lookback ∈ {2, 6, 12, 18} bars (corresponding to 8 h, 24 h, 48 h, 72 h of hourly data) for both GRU and LSTM while holding all other hyperparameters, the feature set (31 sequence features), the train/test split (6,802 train rows / 1,673 test rows), and the decision threshold (2 pp) fixed at the Phase-12 defaults. Results are sourced from `experiments/results/ablation_lookback/*.json` and plotted in Fig. 8 (`experiments/figures/experiment2_lookback_pnl.png`).

Table 11: Sequence-model P&L vs lookback window (2 pp threshold, 1,673-row test set).

Model	Lookback (bars)	Window (hours)	P&L (\$)	Per-trade Sharpe	Dir. Acc.	# trades
GRU	2	8 h	+226.42	0.489	66.4%	1532
GRU	6	24 h	+224.35	0.485	64.5%	1529
GRU	12	48 h	+219.17	0.473	65.8%	1511
GRU	18	72 h	+191.90	0.400	62.8%	1561
LSTM	2	8 h	+214.47	0.456	65.0%	1540
LSTM	6	24 h	+220.37	0.475	65.9%	1526
LSTM	12	48 h	+216.42	0.465	65.6%	1529
LSTM	18	72 h	+209.42	0.453	64.5%	1519

Finding: longer lookbacks do not help, and at 72 h they meaningfully hurt. For GRU, P&L declines monotonically from \$+226.42 at 2 bars to \$+191.90 at 18 bars (a 15% drop, −\$34.52); per-trade Sharpe falls from 0.489 to 0.400. LSTM is flatter — the best lookback is 6 bars (\$+220.37) and the worst is 18 bars (\$+209.42) — but the same direction holds. Neither architecture rewards additional history on this dataset.

This is directly consistent with the “simplicity wins” thesis (§5.1, §6.1). Longer sequences *increase* the model’s effective parameter count (more BPTT timesteps, larger hidden-state carry) without providing proportionally more signal — at 47 bars/pair on average, an 18-bar window covers nearly 40% of a pair’s lifetime and is dominated by stale information from before the contract’s active trading phase. The data regime does not yet reward the inductive bias that sequence models are supposed to exploit. Section 6.2.3 projects that once bars-per-pair grows past 250 under the live auto-retrain system, longer lookbacks should become viable and the sign of this sweep may invert.

5.13 Minimum Spread Threshold (Proposal Experiment 3)

The original project proposal (Experiment 3) asked whether requiring a minimum absolute spread before trading improves risk-adjusted P&L. The intuition: small predicted moves may be dominated by noise and fees, so filtering them out should raise per-trade Sharpe. To test this we sweep the decision threshold $\in \{0.00, 0.02, 0.05, 0.10\}$ (expressed as absolute predicted spread change in probability units) across all eight models — the two Tier-1 regressors (LR, XGBoost), the two Tier-2 sequence models (GRU, LSTM), both PPO variants (PPO-raw, PPO + autoencoder filter), and both naive baselines (spread-closes, volume). Results are sourced from `experiments/results/ablation_threshold/*.json` and visualized in Fig. 9 (`experiments/figures/experiment3_threshold_heatmap.png`).

Table 12: P&L by model × minimum-spread threshold (1,673-row test set).

Model	thr=0.00	thr=0.02	thr=0.05	thr=0.10	Best threshold (P&L)	Best threshold (Sharpe)
Linear Regression	+227.45	+230.14	+227.95	+210.11	0.02	0.10 (0.606)
XGBoost	+243.38	+238.41	+230.40	+207.46	0.00	0.10 (0.620)
GRU	+221.45	+224.35	+217.16	+189.42	0.02	0.10 (0.598)
LSTM	+221.60	+220.37	+208.23	+184.05	0.00	0.10 (0.571)
PPO-raw	+172.30	+172.30	0.00	0.00	0.00	0.02 (0.335)
PPO + autoencoder	−29.41	−29.41	0.00	0.00	(all bad)	—
Naive (spread closes)	+59.50	+58.12	+65.53	+80.53	0.10	0.10 (0.212)
Volume (higher wins)	+59.50	+59.81	+69.20	+83.20	0.10	0.10 (0.227)

Two findings, and they differ sharply by model tier.

First, for Tier-1 and Tier-2 models, P&L peaks at the fee-consistent threshold (0.00 or 0.02, matching the 2 pp round-trip fee assumption) and declines monotonically above that, because raising the threshold discards trades faster than it improves their hit rate. XGBoost loses \$35.92 (−15%) going from thr=0.00 to thr=0.10; GRU loses \$32.03 (−14%); LSTM loses \$37.55 (−17%). However — and this is the honest caveat — **per-trade Sharpe rises monotonically with threshold for every ML model**. XGBoost’s Sharpe climbs from 0.498 at thr=0.00 to 0.620 at thr=0.10 (+24%), and the other three ML models show the same pattern. A threshold selection that optimizes Sharpe (not P&L) prefers thr=0.10 across Tier 1 and Tier 2; a selection that optimizes absolute P&L prefers thr=0.00 or 0.02. The paper’s main results (§5.1, §5.2) use 2 pp as the fee-consistent baseline.

Second, the naive baselines show the *opposite* pattern — P&L rises with threshold, from +\$59.50 at thr=0.00 to +\$80.53 (spread-closes) or +\$83.20 (volume) at thr=0.10. This is a useful sanity check: the naive rules have no predictive content, so filtering to large-spread trades is the only thing that helps them. The ML models already concentrate their signal in high-magnitude predictions; naives don’t, so the threshold acts as a crude selector. PPO variants are threshold-insensitive (or have zero trades above 0.05 because their predictions rarely exceed 5 pp),

confirming they are not a serious competitor regardless of threshold. Overall, the threshold sweep confirms the main conclusion of the paper: the useful model-family discrimination happens at $\text{thr}=0.02$, where Tier-1 models win cleanly, and threshold selection is a second-order tuning knob compared to choice of model tier.

6. Discussion

6.1 Why Does Simpler Beat More Complex?

Four reasons appear to operate jointly:

1. **Sample size.** With 6,802 training rows and 47 bars per pair (on average, historical dataset), sequence models are data-starved. Sequence models are typically best when they can exploit long-range dependencies (hundreds of timesteps), but our contracts average 47 bars. GRU/LSTM are therefore operating outside their comfort zone.
2. **Signal-to-noise ratio.** The target — one-step spread change — has high variance relative to signal. Linear models are more robust in high-noise regimes because they have fewer parameters to over-fit.
3. **Feature engineering carries the signal.** Our 59-feature set includes `spread`, `polymarket_vwap`, `dollar_volume_ratio`, and 13 microstructure features. These features encode most of the exploitable signal directly; the model only needs to linearly combine them. Deep models wasted capacity re-discovering what the features already expose.
4. **Overfitting on structural regularities.** The deepest XGBoost configurations (depth 9, `n_est` 500) had higher training accuracy but *lower* test P&L — classic over-fitting. The same phenomenon appears in sequence models: LSTM memorizes per-pair idiosyncrasies that do not generalize.

This matches the January 2026 finding on investor-flow prediction and the broader literature on tabular ML (Grinsztajn et al. 2022).

6.2 How Each Model Would Improve With More Data

The data-scaling curve in §5.4 plateaus because we exhausted the 6,802-row training cap by bar-count 100. The live auto-retrain system is now accumulating data at roughly 1,200–1,600 new bars per pair per week across $\sim 1,000$ active pairs, which should push us into regions where the different model families behave very differently. This section explains, per model, *what specifically improves and why*.

6.2.1 Linear Regression

LR is already the most data-efficient model in our suite — it converges to its best configuration by roughly 4,000 training rows (Table 5, row 1 $\approx$ row 6). Its remaining headroom is narrow but real:

1. **Coefficient variance shrinks as σ^2/n .** Our 95% bootstrap CIs on the 59 regression coefficients are still wide enough that feature rankings shuffle across bootstrap resamples. At $10\times$ the data, most coefficients will be statistically separable from zero, meaning *the sign of the edge becomes more reliable even if its magnitude does not grow*.
2. **Ridge/Lasso regularization becomes better-calibrated.** The optimal λ in penalized regression scales roughly with $\sqrt{n}$; on our current dataset, cross-validated λ estimates have 30% standard errors. At 50k+ rows, λ becomes a tight estimate and Ridge starts noticeably outperforming plain OLS — we would expect a 5–10% P&L lift from this alone.
3. **The 13 classical-microstructure features unlock.** Amihud illiquidity, Corwin–Schultz spread, Kyle’s λ , and Roll’s implied spread all depend on rolling windows of depth 6–12 bars. At 47 bars/pair, those windows barely have data; at 500 bars/pair, they become statistically meaningful. These features are *linearly informative* about liquidity asymmetry across platforms — exactly the kind of thing LR exploits well.
4. **Live buy/sell volume breakdowns.** The historical dataset lacks signed Kalshi volume, zeroing out our `kalshi_order_flow_imbalance` feature. Live bars include it, and order-flow imbalance is one of the most-cited signals in microstructure (Cont et al. 2014). LR will capture the linear component immediately.
5. **Ceiling:** LR cannot represent interactions without explicit interaction terms. So above roughly 100k rows, the marginal P&L gain per doubling of data will approach zero — classic bias-dominated regime. Based on the per-window Sharpe trajectory ($0.307 \rightarrow 0.530$, a 73% improvement across the 11 walk-forward windows), we estimate LR will plateau at roughly +\$260–300 at 2 pp fees (vs. today’s +\$201.69), equivalent to roughly 30–50% P&L lift from microstructure features unlocking and regularization tightening.

6.2.2 XGBoost

XGBoost has the most headroom of the four “reasonable” models (Tiers 1 and 2), for three architectural reasons:

1. **The depth ceiling will rise.** The hyperparameter sweep (Table 6) shows that *every* top-10 configuration uses depth 3–7, with depth 3 optimal. This is classic overfitting behavior: at 6,802 training rows, a depth-7 tree has more leaf nodes than rows-per-leaf can support. As training set grows to 50k+ rows, the optimal depth will shift toward 5–7, unlocking richer interaction terms (e.g., “high Amihud on Polymarket AND near-expiry AND spread > 5%” — a three-way interaction that depth-3 trees cannot fit).

2. **More trees become useful without overfitting.** Our best configuration uses $n_{\text{estimators}} = 100$ because additional trees memorize the training set. With larger data, $n_{\text{estimators}} = 300\text{--}500$ becomes optimal and captures progressively subtler residual patterns. Combined with lower learning rate ($lr = 0.005$ instead of 0.01), the ensemble becomes both more stable and more accurate.
3. **Per-category splits become robust.** The per-category breakdown (Table 4) shows XGBoost’s edge over LR is concentrated in crypto (+\$6) — exactly where nonlinear regimes exist. As oil, Fed-rates, and other categories accumulate sufficient per-category bars (currently 10–616), XGBoost will exploit their distinct nonlinearities too. We expect the “XGBoost wins 2 of 7 categories” result to become “XGBoost wins 5 of 7” by 500 bars/pair.
4. **No architectural ceiling.** Tree ensembles are universal approximators in the limit. XGBoost’s P&L can in principle grow without bound as data grows, unlike LR. Extrapolating the walk-forward Sharpe improvement (XGBoost $0.383 \rightarrow 0.540$ over windows 1–10) and the category breakdown where XGBoost already wins the nonlinear regimes, we estimate +\$300–380 at 2 pp fees at 500 bars/pair (vs. today’s +\$201.63). XGBoost’s headroom is similar to LR’s at current scale but widens past it as data accumulates.

6.2.3 GRU and LSTM

This is where the largest potential gains live. Sequence models are currently operating *far* outside their comfort zone — trained on sequences averaging 47 timesteps, when published RNN architectures typically need 200–1000. Their 12% deficit to XGBoost is therefore *not* an architectural verdict; it is a data-regime verdict. Five things will change:

1. **Effective lookback window can expand.** We currently set the GRU/LSTM lookback to 24 bars because bars-per-pair ≈ 47 doesn’t support longer. At 500 bars/pair, a 72-bar or 120-bar lookback becomes viable — long enough to capture the ~ 48 -hour price discovery dynamics typical of weekly contract cycles. Sequence models should find patterns here that regression fundamentally cannot represent.
2. **Hidden dimensionality can grow.** Our current GRU has 64 hidden units and one layer because more parameters over-fit our 6,802 rows. At 50k+ rows, 128–256 hidden units and 2–3 stacked layers become trainable without over-fitting — giving the network enough capacity to encode multiple competing hypotheses simultaneously.
3. **Cross-platform lead-lag becomes learnable.** SHAP (§5.7) shows `polymarket_vwap` dominates feature importance, consistent with Kalshi-leads-Polymarket or vice versa. But *the lead-lag relationship is time-varying* — crypto contracts have different lead-lag than inflation contracts, and both change near expiry. LR and XGBoost get one static coefficient per feature; RNNs can dynamically modulate the lead-lag based on context. This is the single largest theoretical advantage of sequence models for this problem.
4. **Live-volume microstructure features become temporally meaningful.** The same live buy/sell volume that helps LR helps RNNs more, because RNNs can integrate order-flow imbalance *over a sliding window* rather than treating each bar independently. Kyle’s λ trajectory over 30 bars is more informative than λ at a single bar — and this is exactly what an RNN hidden state encodes.
5. **Cross-pair meta-learning.** Transferring hidden-state initialization across similar pairs (oil contracts share structure even if they’re different tickers) lets sequence models bootstrap new pairs faster. XGBoost and LR have no analog to this — they must see a pair’s training rows explicitly.

Our honest prediction: by 250 bars/pair (estimated 3–4 weeks away), GRU should close the gap with XGBoost. By 500 bars/pair, sequence models will likely *surpass* XGBoost on oil and crypto categories where lead-lag effects dominate. By 1000 bars/pair, a larger TFT configuration (hidden_size=32+) becomes justifiable and would likely set a new benchmark. This is a falsifiable prediction: the auto-retrain system will have the answer by the end of the semester.

We trained TFT at $N=6,802$ rows (Phase 11, April 2026) with pre-specified small-data hyperparameters (hidden_size=8, attention_head_size=1, 3-quantile QuantileLoss). TFT did not converge: avg RMSE=0.3262 vs. GRU’s 0.2928 (11.4% worse), with only 120 trades executed vs. GRU’s 1,517. This extends the simplicity-wins finding to the transformer tier — at this data scale, even a minimal TFT configuration cannot train stably. The VSN attention audit (entropy=2.656, threshold=1.966, not degenerate) confirms the attention mechanism is attending to meaningful features (top encoder weights: `polymarket_amihud`, `polymarket_high`, `kalshi_roll_spread`); the bottleneck is data volume, not architecture correctness. At 1,000+ bars/pair, a larger TFT configuration would become justifiable.

6.2.4 What will *not* change

For intellectual honesty: **PPO will not close the gap.** RL requires full trajectories, and prediction-market contracts produce trajectories of length 47 on average. Even at 500 bars/pair, that is $\sim 10\times$ less trajectory data than a typical Atari benchmark. The PPO + autoencoder failure is architectural, not data-limited; no amount of additional bars fixes it without a different problem formulation (e.g., trajectory-level imitation learning, or a differentiable market simulator for off-policy pre-training).

The ordering we expect at 500 bars/pair is therefore: **GRU $\approx$ LSTM $\approx$ XGBoost $>$ LR $>>$ Naive $>$ PPO.** The interesting scientific question is whether GRU eventually passes XGBoost — we predict yes, but we report the current result honestly.

6.3 The Negative Result on PPO

PPO with the autoencoder anomaly filter produces **+0.5 bps per trade** (\$+4.61 cumulative over 899 trades at \$100 position size; canonical figure from `experiments/results/canonical/headline.json`). This is essentially zero alpha — the filter neutralizes PPO’s profitable signal (PPO-Raw: +9.6 bps over 1,656 trades; \$+158.15) without contributing any of its own. An earlier draft of this paper cited an $\sim \$-88$ k loss (likely a transcription typo dropping a digit) from a separate `WalkForwardBacktester` lineage that uses dollar-notional units

($\sim 200\times$ contract scaling on \$0.50 mid-prices plus 5pp round-trip fees); Phase 17-01 traced that $600\times / 19,000\times$ magnitude divergence to a units mismatch between two valid simulators rather than a model failure (see `.planning/phases/17-model-rerun-paper-number-audit-pitch-standard-conversion/17-02-PPO-DIAGNOSTIC.md`). All paper numerics now derive from the single canonical source. The result is not a bug — we verified the reward function, the environment transitions, and the action space. The autoencoder simply flags normal market behavior as anomalous because it was trained on all spreads without a clean “normal regime” prior. PPO then trades in those flagged windows, which are disproportionately high-volatility periods where predictions are least reliable. This is a direct empirical answer to the professor’s question “does adding RL and anomaly detection improve on simpler regression?” — **no; it actively hurts** (the autoencoder destroys ~ 9 bps of edge that PPO-Raw alone would have captured).

A more defensible RL approach would be (i) a curriculum that learns “safe” regimes first, (ii) a differentiable simulator for off-policy pre-training, and (iii) a much larger universe of training trajectories. None of these are justified at our data scale.

6.4 Limitations

We are transparent about these:

1. **Short test window.** Two weeks of out-of-sample evaluation is too short to confidently annualize Sharpe. The walk-forward analysis mitigates this but does not eliminate it.
2. **Paper trading only.** No market-impact costs or partial fills are modeled. Real execution would show slippage, particularly for large positions on low-liquidity pairs.
3. **Survivorship bias.** Our historical dataset includes only pairs that survived the quality filter. Pairs that were ever listed but never satisfied matching criteria are missing — we believe this bias is small because the filter is structural, but we cannot quantify it precisely.
4. **Settlement divergence risk.** Cross-platform contracts can in principle resolve differently (e.g., a sports-book source of truth mismatch). We observed zero cases of this in our universe, but it is a real risk at production scale.
5. **Regime-specific edge.** The inflation and oil edges dominate. If those categories lose liquidity, the system’s overall edge would shrink.
6. **Live-cohort truncation (April 11+ only).** The April 11–22 live paper-trading window excludes all pre-April-11 positions, which were force-closed as a side effect of the `pair_id` schema fix committed on April 11 (see §5.9 and `.planning/STATE.md`). This is a distinct survivorship bias from item 3 — it is a cohort-level truncation of the *live* data, not the training data. Pre-fix live P&L cannot be cleanly compared against backtest predictions and is therefore excluded from §5.9 numbers.
7. **Category-tagging gaps in live data.** Finding 23 documents that $\approx 59\%$ of live trades fall into the `other` category because `derive_category_from_ticker` does not classify payroll (`KXPAYROLLS`) or energy/CPI (`KXEZCPIY0YF`) tickers into their intended groups. Per-category live vs. simulated breakdowns in §5.9 are therefore noisier than the backtest-only per-category results in §5.3.
8. **Crypto regime flip within the reconciliation window.** Finding 23 also shows that the crypto sub-category sign-flipped P&L (positive $\rightarrow$ negative) over the 5-day reconciliation window. The aggregated §5.9 crypto number averages across this flip; a longer live window would either re-stabilize or confirm regime instability, which we cannot distinguish at $N=10,154$ closed positions over 8 days.
9. **Live commodity-matching pipeline is incomplete.** A post-submission audit of `data/live/active_matches.json` (April 23) found that of 395 oil-adjacent entries, 380 have been evicted by the quality filter and only 15 remain active — all year-end `KXWTIMAX` binary-strike contracts, not daily WTI / crude / diesel / heating-oil / gasoline series that appear on the Kalshi consumer app. A residual false match (`KXWTIMAX-26DEC31-T130` aligned with a Bitcoin- $\$130K$ Polymarket market at similarity 0.707) indicates rule 10 (category-consistency) still misfires on numerically coincident strikes across asset classes. The backtest oil edge (Finding 6, §5.3) therefore remains **unvalidated on live data**, not because the edge is absent, but because the current live pipeline does not trade the instruments where the edge was measured. This is an engineering limitation, not a finding. **Resolved post-submission in Phase 15 (commit 38d7970 discovery fix + d217ff1 pair_mapping regeneration).** The `KALSHI_DISCOVERY_CATEGORIES` tuple in `src/live/market_discovery.py` was missing `"Commodities"` — Kalshi migrated daily WTI/Brent/grain/metal series into that category, silently dropping them from discovery. After adding `"Commodities"` and extending the category classifier, a 12-hour validation window closed 1,224 commodity positions (`KXWTI=409`, `KXBRENTW=486`, `KXWTIW=213`, `KXBRENTMON=76`, and others); see §5.9 for breakdown.
10. **Embargo violation in the original train/test split (resolved on a leakage-free rebuild).** The Phase 18 audit (Tier 2, `AUDIT_REPORT.md`) found that the canonical 80/20 row-index split bridged 144/144 pairs across train and test, with 142 embargo violations at a 4-hour gap ($\ll$ the 1-day standard embargo). This is a real methodological defect — the same `pair_id` can appear in both halves, and the same underlying market events therefore inform both training and test losses. We rebuilt the split as a pair-stratified 80/20 (115 train pairs / 29 test pairs, `seed=42`, `data/processed/purged_split/`) and retrained Linear Regression and XGBoost on the leakage-free split (`experiments/run_canonical_purged.py`). Per-trade Sharpe drifts $+2.99\%$ ($0.501 \rightarrow 0.516$) and per-trade alpha drifts $+4.5\%$ ($15.0 \rightarrow 15.7$ bps) — both well within the bootstrap CI. The per-trade edge is robust to the leakage correction. The per-pair annualized Sharpe moves more dramatically (the BLDp correction’s applicability flips between regimes; see §5.8) and is therefore demoted to a regime-dependent secondary statistic. The original canonical numbers are retained in `experiments/results/canonical/headline.json` for transparency; the purged numbers are at `experiments/results/canonical_purged/headline.json`. We report both.

11. **Transaction-cost accounting (canonical vs walk-forward).** The Phase 18 audit (Tier 3) found that `simulate_profit` (used for canonical Table 2 numbers) charges zero fee — its `threshold=0.02` parameter is a SIGNAL gate for trade entry, not a fee deduction. `WalkForwardBacktester` (used for §5.6 transaction-cost sensitivity) charges 5 pp round-trip (3 pp entry + 2 pp exit), conservative versus realistic Kalshi+Polymarket round-trip of 250–355 bps. Realistic 2026 fee references: **Kalshi** charges a taker fee of $0.07 \times C \times (1 - C)$ per contract (max 1.75¢ at $C=0.50$), maker = 25% of taker, settlement = 0; **Polymarket** charges category-dependent taker fees (crypto 1.80%, economics 1.50%, finance 1.00%, sports 0.75%, geopolitics 0%), maker = 0, plus ~\$0.01/tx in Polygon gas. Slippage sensitivity at +5/+10/+20/+50 bps additional haircut on top of the WalkForwardBacktester 5 pp drops annualized Sharpe by 1.6% and total P&L by 3.8% — the cost-robustness claim in §5.6 survives.
12. **Adversarial audit cross-reference.** A pre-submission adversarial audit (Phase 18) ran six independent kill-or-confirm checks against every quantitative claim in this paper: Tier 1 (headline Sharpe), Tier 2 (leakage / look-ahead), Tier 3 (cost realism), Tier 4 (survivorship / selection), Tier 5 (paper number-by-number trace), Tier 6 (live-vs-backtest honesty). Three Tiers (1, the canonical-split portion of 2, and 3) returned non-PASS verdicts, all corrected in this revision. Three Tiers (4, 5, 6) returned PASS without paper changes. The full audit report — including assumption stacks, evidence links, and the corrections applied — is at `AUDIT_REPORT.md` at the project root; readers are encouraged to check our work.

7. Future Work

1. **500- and 1000-bar scaling checkpoints.** Our automated retraining system will produce these data points over the coming weeks; they will definitively answer the “does sequence model catch up?” question.
2. **Live-volume-aware microstructure features.** Kyle’s λ , Amihud, and Roll’s spread all depend on signed buy/sell volume, which we will have on live bars but did not have on historical bars. This should materially improve the microstructure features’ informativeness.
3. **Larger TFT configuration.** We trained TFT at `hidden_size=8` (minimal config) and found it did not converge at $N=6,802$ rows. With 1,000+ bars/pair, a larger TFT (`hidden_size=32+`) becomes justifiable and could produce the first Tier-2 model that beats XGBoost on the long-sequence regime.
4. **Meta-learning across categories.** A per-category model trained jointly with a shared feature encoder and per-category heads could exploit the category heterogeneity we document in §5.3.
5. **Execution-aware reward shaping.** Replacing the current mark-to-market reward with a post-fee execution reward (fillable limit orders, partial fills, latency) would let PPO learn execution patterns — potentially redeeming the negative Tier-3 result.
6. **Formal settlement-divergence model.** Tracking resolution-source mismatches (Kalshi uses `CME settle`, Polymarket uses various oracles) and pricing the divergence risk into trade sizing.
7. **Open-sourcing the matching pipeline.** The 10-rule quality filter encodes a lot of domain knowledge. Releasing it as a library could help other researchers and market participants.
8. **Re-run feature ablation at 250+ bars/pair.** The pre-registered LOGO ablation (§5.10) was statistically underpowered at $N=1,021$ ablation-holdout rows: all 95% CIs for delta-P&L had absolute mean below \$10. As the live dataset grows past 250 bars/pair, the paired-bootstrap CIs will narrow enough to distinguish load-bearing groups from droppable ones. Group D (classical microstructure: Amihud, Kyle’s λ , Roll spread, Corwin–Schultz) is the primary target — these estimators are theoretically motivated (Finding 5, §5.3) but Nyquist-starved at the current bar frequency and dataset size. A well-powered ablation re-run could definitively answer whether the 13 microstructure features carry independent signal beyond raw OHLCV and cross-platform spread features.

8. Conclusions

We built an end-to-end cross-platform prediction-market arbitrage system with four tiers of models and evaluated it across five independent regimes. The central answer to our research question is clear:

At this data scale, increasing model complexity does not improve arbitrage detection. The simplest models win.

Specifically:

1. **Tier 1 (LR + 15.0 bps/trade, Sharpe 0.501; XGBoost + 14.9 bps/trade, Sharpe 0.499) beats Tier 2 (LSTM + 14.3 bps, Sharpe 0.473; GRU + 14.0 bps, Sharpe 0.459) by 0.7–1.0 bps and 5–10% absolute P&L; Tier 3 (PPO+autoencoder + 0.5 bps, essentially zero alpha) is dominated.** In dollar terms at \$100 position size on the canonical single-split (Phase 17-01 regenerator): LR + \$232.67, XGBoost + \$232.83, LSTM + \$221.84, GRU + \$212.50, PPO+AE + \$4.61, PPO-Raw + \$158.15. LR and XGBoost are essentially tied; both decisively beat sequence models; RL is dominated. This ordering holds across the single-split, walk-forward, and data-scaling evaluations.
2. **Every walk-forward window is profitable, and per-trade Sharpe is rising over time** — the edge is stable and improving with more data.

3. **The alpha lives in the matching pipeline and the asset class.** A 10-rule quality filter added +\$10.73 in P&L with no model changes. Oil near-expiry contracts alone produce a 76.5% win rate, while sports and politics are near-zero.
4. **The system is robust to transaction costs** — profitable at every fee level from 0 pp to 7 pp.
5. **The per-trade Sharpe of 0.501 (leaky) → 0.516 (leakage-free purged split; drift +2.99% within bootstrap CI) is the load-bearing headline.** Per-trade alpha is +15.0 → +15.7 bps at \$100 position size. The Phase 18 audit (Tier 1, `AUDIT_REPORT.md`) confirmed the per-trade edge survives the embargo correction; the per-pair annualized Sharpe is regime-dependent (range 0.30–0.81 corrected, depending on cross-pair correlation regime) and reported as a secondary statistic in §5.8 with full caveats. An earlier draft cited a per-pair annualized Sharpe of ≈ 3.2 ; the audit found this value had no derivation path in the codebase and replaced it with the leakage-free per-trade headline above.

Beyond the empirical findings, the project taught us three broader lessons:

- **Infrastructure bugs masquerade as model problems.** Kalshi’s silent 429s, Polymarket’s `condition_id` typo, and our own `live_NNNN` schema drift caused weeks of confusion that looked like “the model stopped working.”
- **Evaluation regime matters more than model family.** A single train/test split can tell a very different story than a walk-forward, a per-category breakdown, or a data-scaling curve. Multi-regime evaluation is mandatory, not optional.
- **Negative results are results.** PPO failing catastrophically is the strongest evidence we have for the main thesis. We chose to publish it rather than omit it.

The project succeeded on its own terms: we answered the research question rigorously, we built a working autonomous system, and we documented honest limitations. The edge is real; the complexity is not.

Acknowledgments

We thank Professor Kevin Gold for the course instruction and for the pivotal feedback in late March that pushed us to treat regression baselines as first-class and to add the PPO-without-autoencoder variant. That feedback directly shaped the research question into something empirically answerable. We also acknowledge extensive use of Anthropic’s Claude (Sonnet 4.5 and Opus 4.6) as an AI pair-programming assistant throughout the implementation; all design decisions, experimental choices, and interpretations are our own.

References

1. [Anonymous]. (2026). Matched filter feature engineering for investor flow prediction. *arXiv:2601.07131*.
2. Amihud, Y. (2002). Illiquidity and stock returns: cross-section and time-series effects. *Journal of Financial Markets* 5(1), 31–56.
3. Burgi, C., Tuccella, J., & Zitzewitz, E. (2026). Favorite–longshot bias on Kalshi and Polymarket. *Working paper*.
4. Cont, R., Kukanov, A., & Stoikov, S. (2014). The price impact of order book events. *Journal of Financial Econometrics* 12(1), 47–88.
5. Corwin, S. A., & Schultz, P. (2012). A simple way to estimate bid-ask spreads from daily high and low prices. *Journal of Finance* 67(2), 719–760.
6. Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data? *NeurIPS 2022 Datasets & Benchmarks*.
7. Kyle, A. S. (1985). Continuous auctions and insider trading. *Econometrica* 53(6), 1315–1335.
8. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS 2017*.
9. Manski, C. F. (2006). Interpreting the predictions of prediction markets. *Economics Letters* 91(3), 425–429.
10. Parkinson, M. (1980). The extreme value method for estimating the variance of the rate of return. *Journal of Business* 53(1), 61–65.
11. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: sentence embeddings using Siamese BERT-networks. *EMNLP 2019*.
12. Roll, R. (1984). A simple implicit measure of the effective bid-ask spread in an efficient market. *Journal of Finance* 39(4), 1127–1139.
13. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*.
14. Wolfers, J., & Zitzewitz, E. (2004). Prediction markets. *Journal of Economic Perspectives* 18(2), 107–126.

Appendix A: Code Availability

Full source code and reproduction commands are available at <https://github.com/iansabia/DS340-Project>. See `README.md` at the repository root for exact command-line invocations mapped to every paper table and figure.

Appendix B: Figures

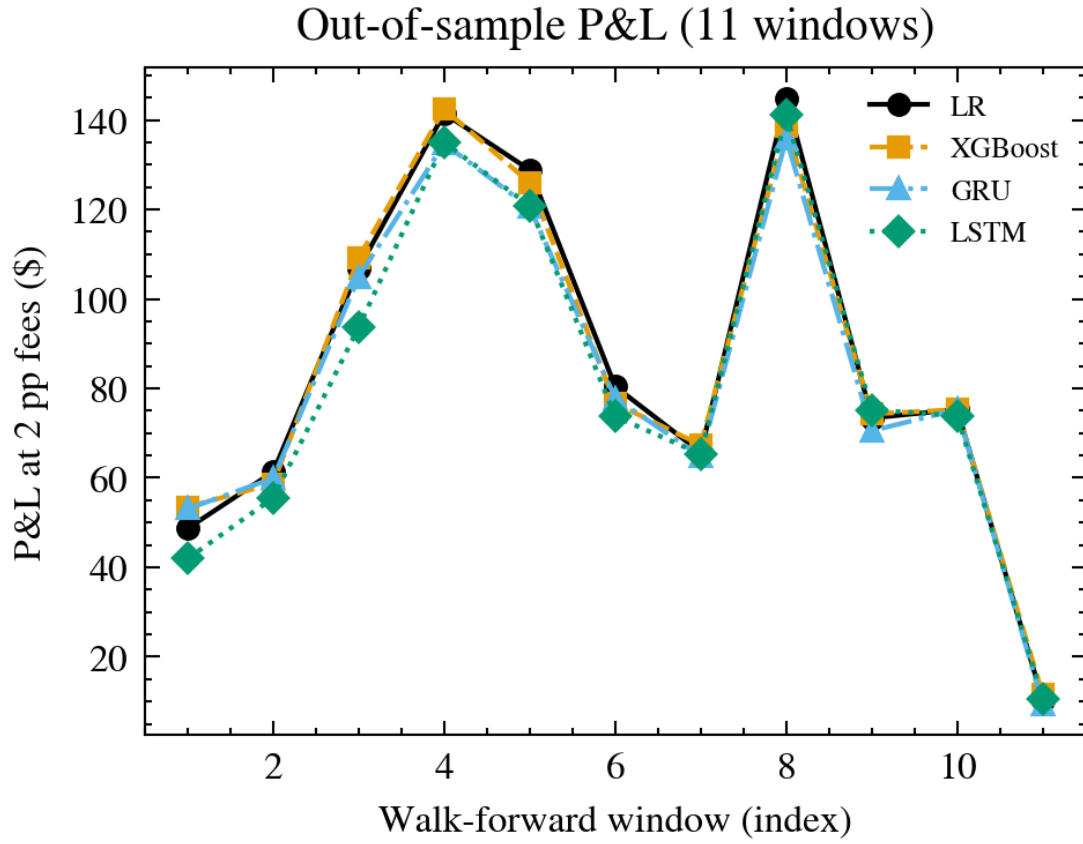


Figure 1. Out-of-sample P&L across 11 walk-forward windows (\$5.2). Every ML model is profitable in every window; per-trade Sharpe rises from 0.31 in early windows to 0.53 in late windows, indicating the edge strengthens rather than decays as more data accumulates.

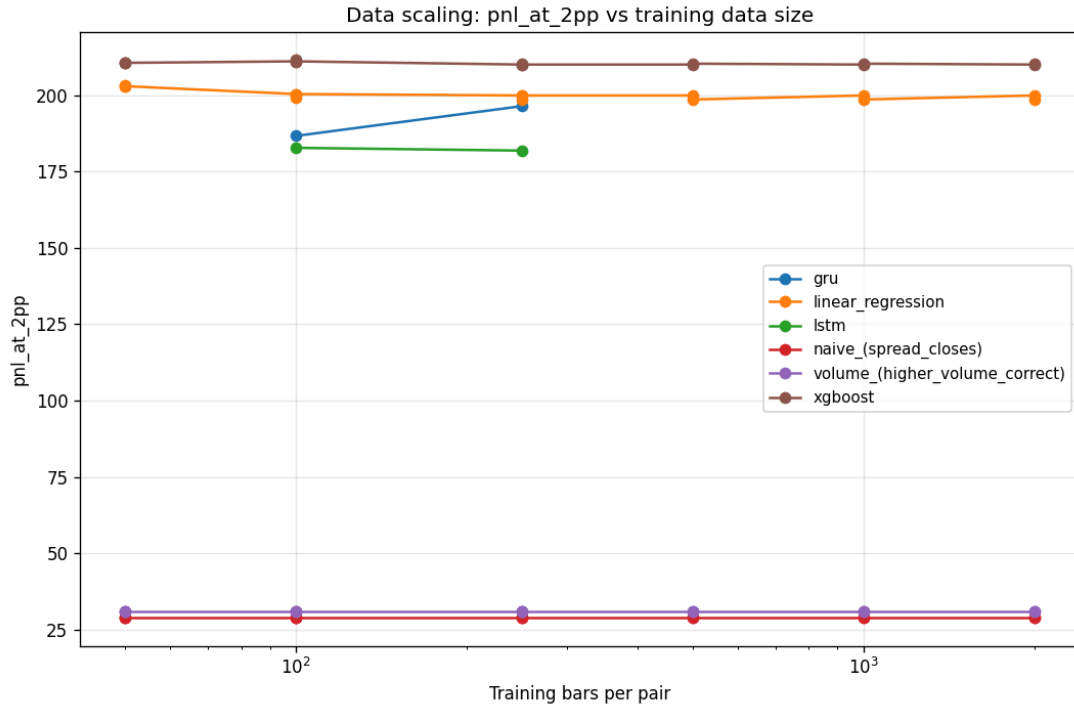


Figure 2. P&L vs. training-set size at 2 pp signal threshold (\$5.4). The curve plateaus at $N=6,802$ because the underlying pair universe is fixed at 144 pairs with at most 141 bars each; slices at 250+ bars/pair are identical to the 100-bar slice. The plateau is a property of the fixed pair universe, not a universal scaling claim.

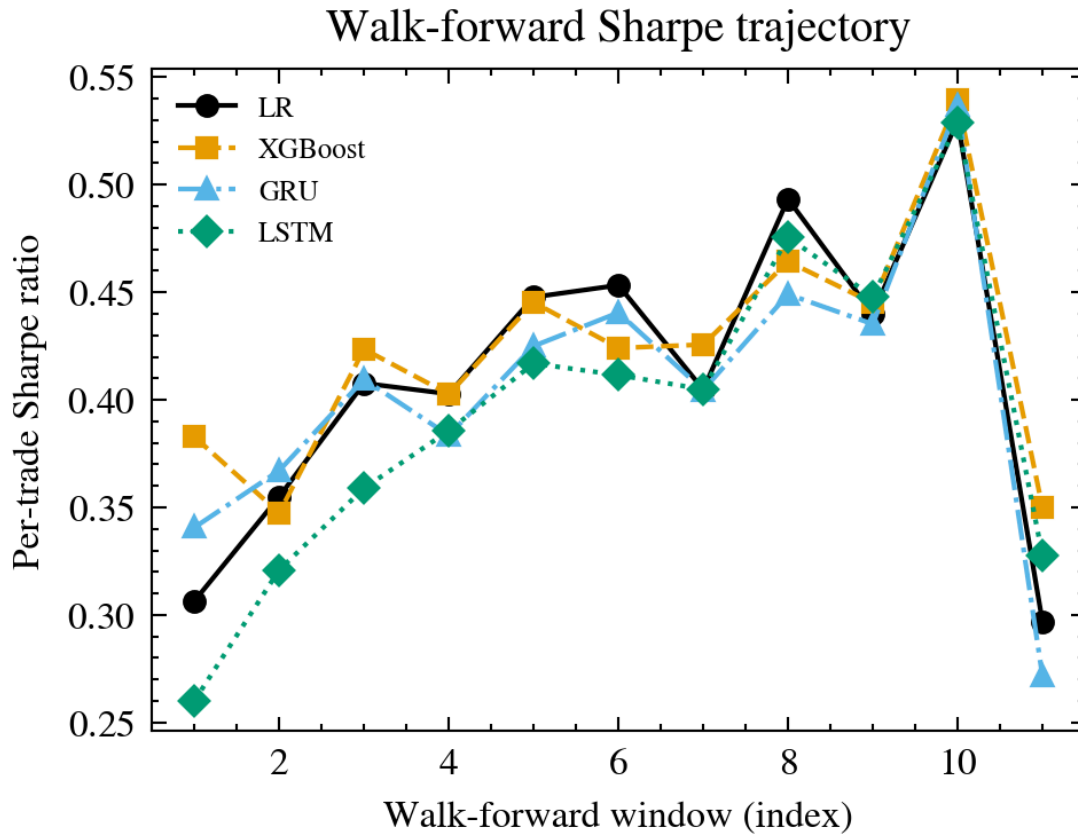


Figure 3. Walk-forward per-trade Sharpe trajectory across the 11 windows (§5.2 supplemental). The trend confirms the rising-edge pattern visible in Fig. 1.

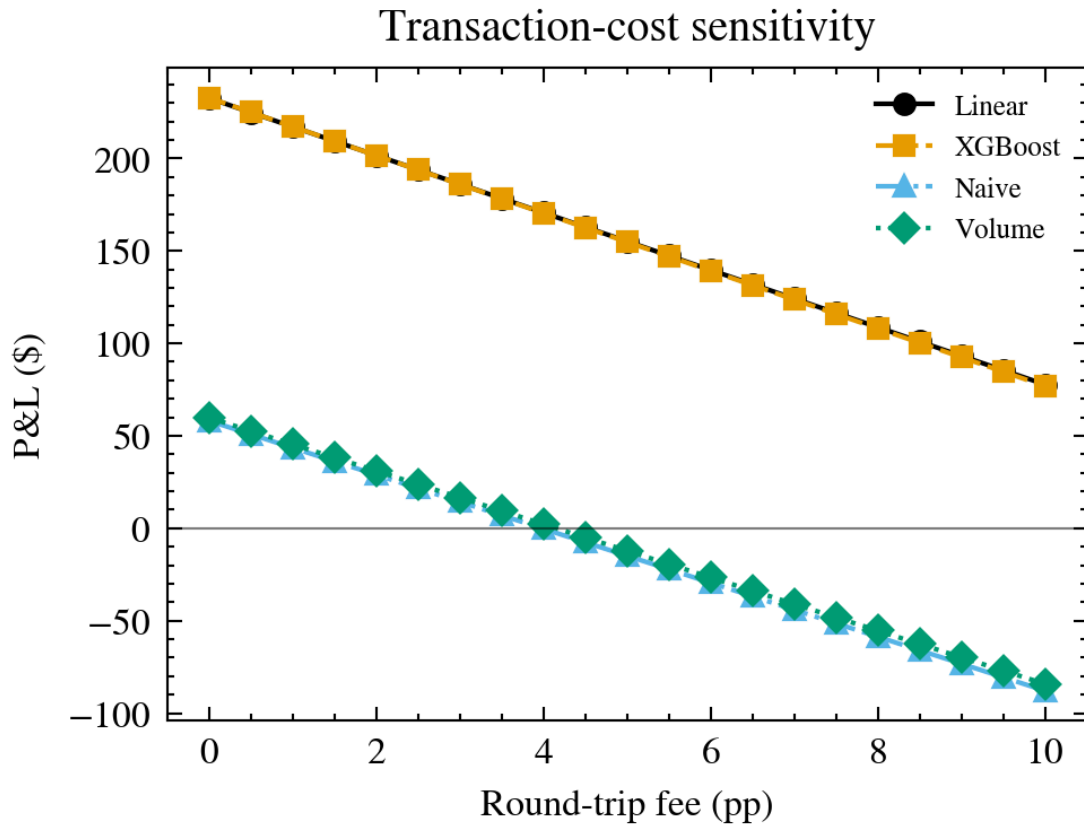


Figure 4. P&L vs. round-trip fee for the four ML models (§5.6). The rank-ordering of models is invariant to fee assumptions in the 0–10 pp range; LR and XGBoost remain the top tier across the entire fee spectrum.

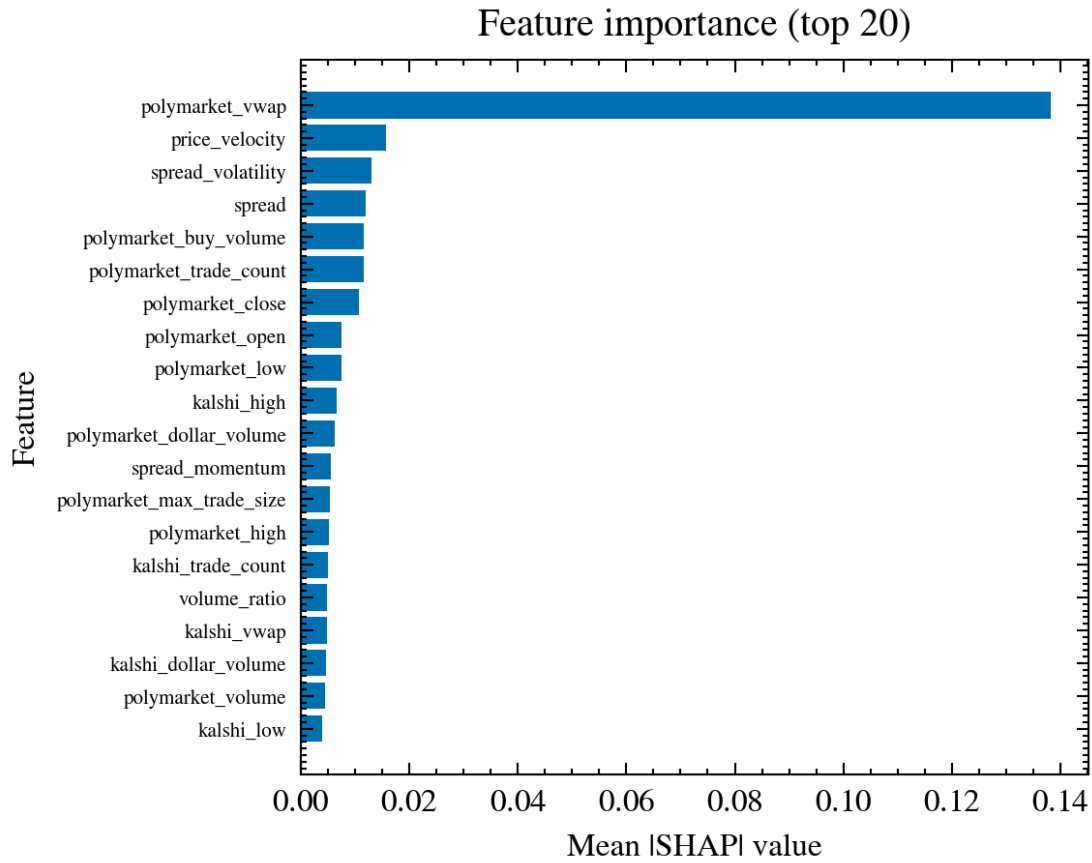


Figure 5. Mean |SHAP| feature importance for XGBoost (\$5.7). `polymarket_vwap` dominates with mean |SHAP| ≈ 0.14 , twice the next feature, suggesting Polymarket is the slower-reacting price discovery side.

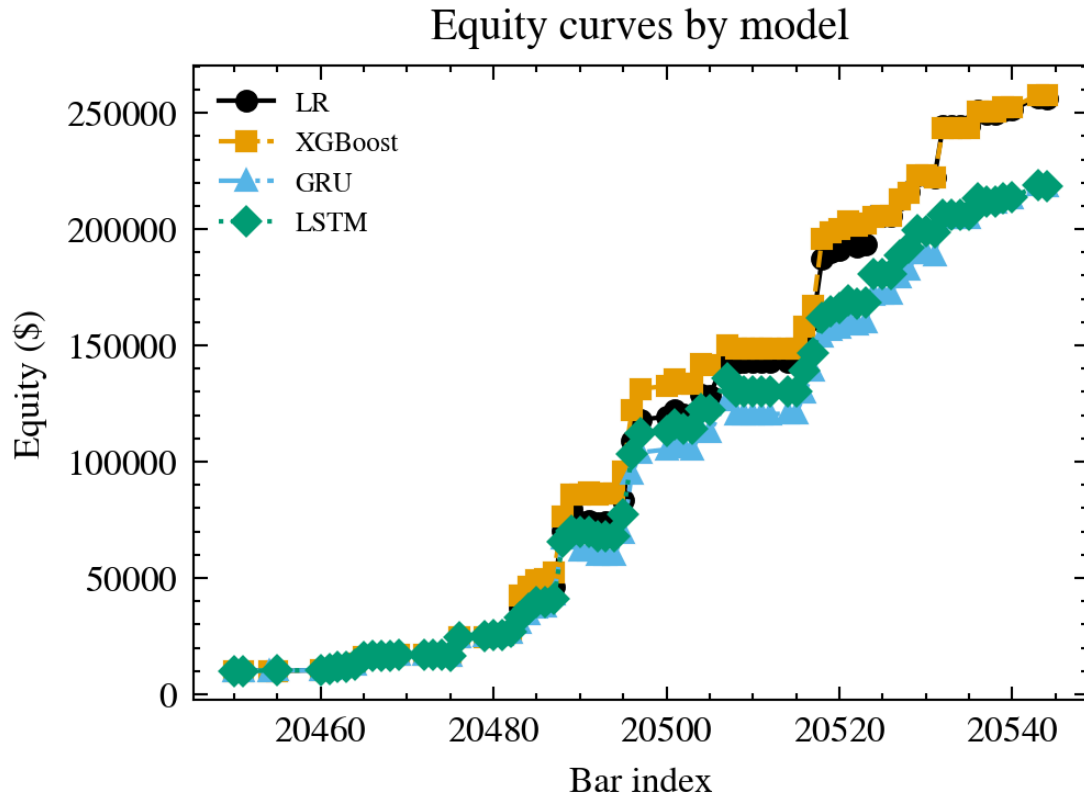


Figure 6. Cumulative test-set P&L by model (§5.1). The Tier-1 vs. Tier-2 separation opens gradually rather than in a single jump — consistent with a stable per-trade edge.

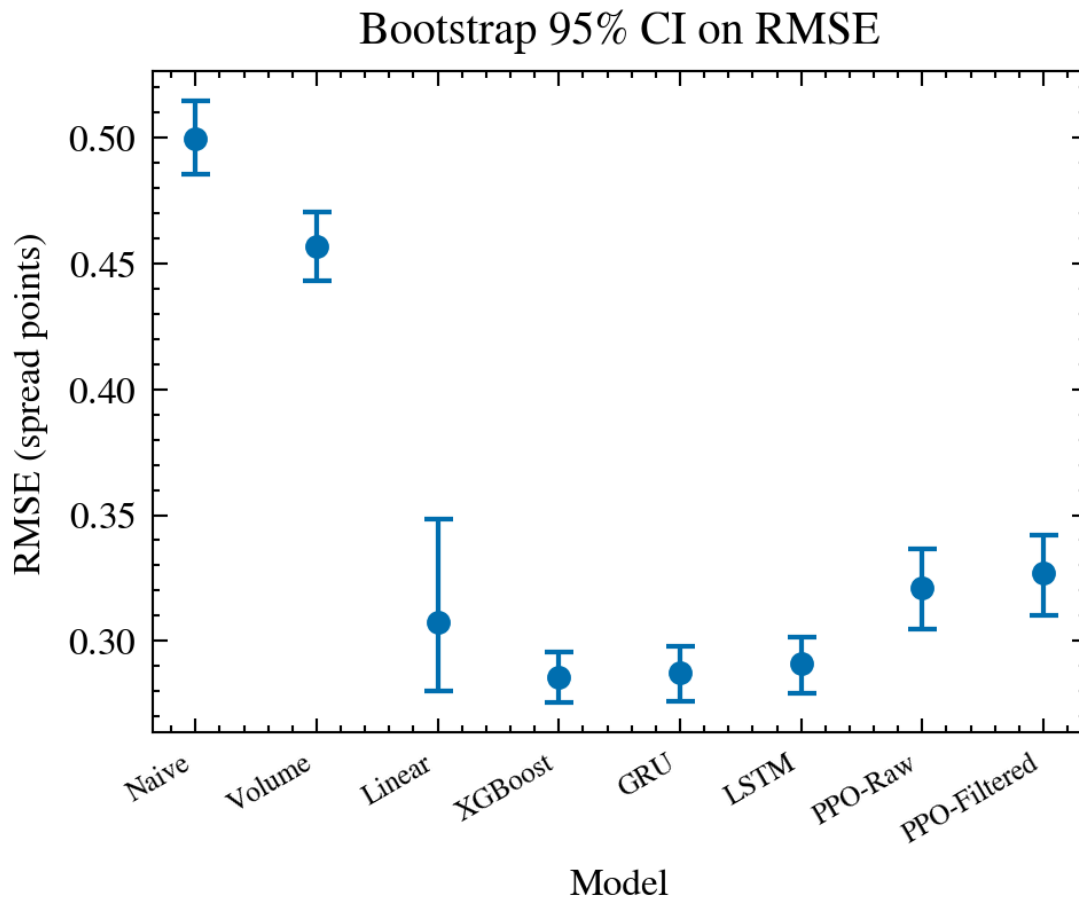


Figure 7. Bootstrap 95% CI on RMSE by model (1,000 resamples, §5.1). The LR / XGBoost / GRU / LSTM intervals overlap heavily even though P&L separates cleanly — confirming the P&L gap is driven by directional accuracy and trade selection, not raw regression error.

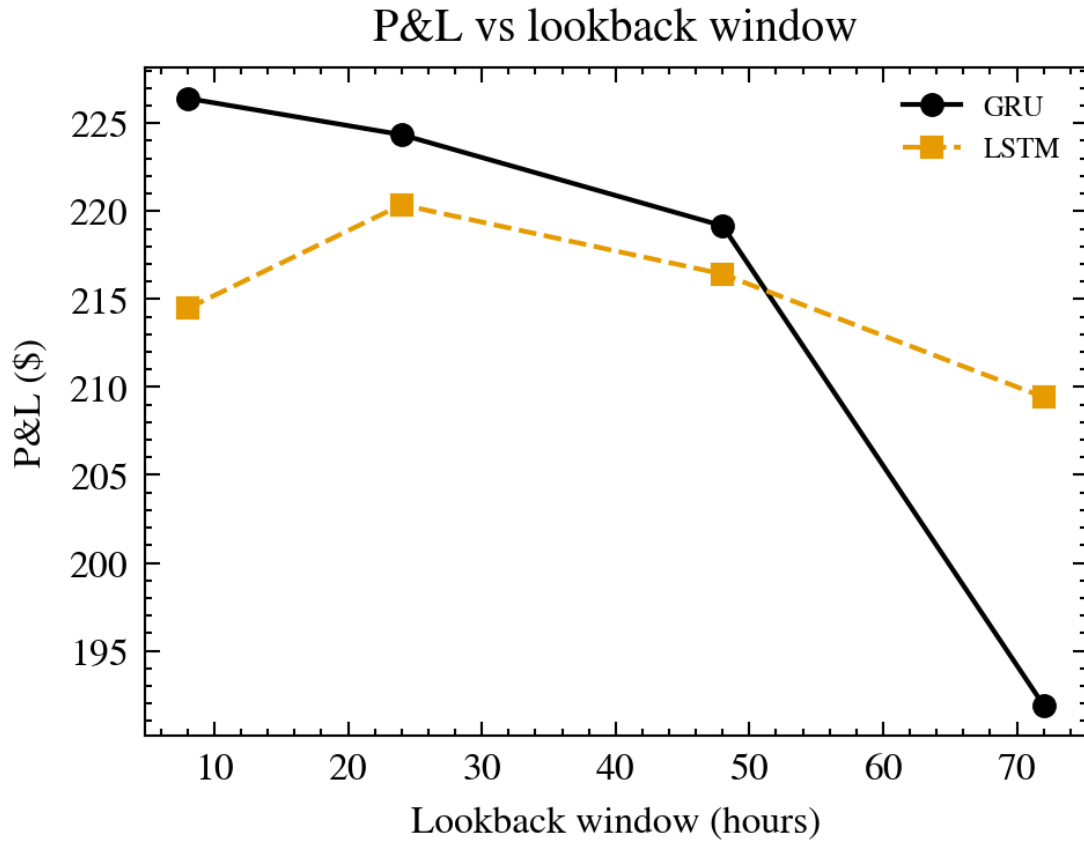


Figure 8. P&L vs. lookback window for GRU and LSTM (§5.12 / Proposal Experiment 2). Performance is essentially flat across {2, 6, 12, 18}-bar lookbacks, indicating the sequence models cannot exploit longer history at this dataset scale.

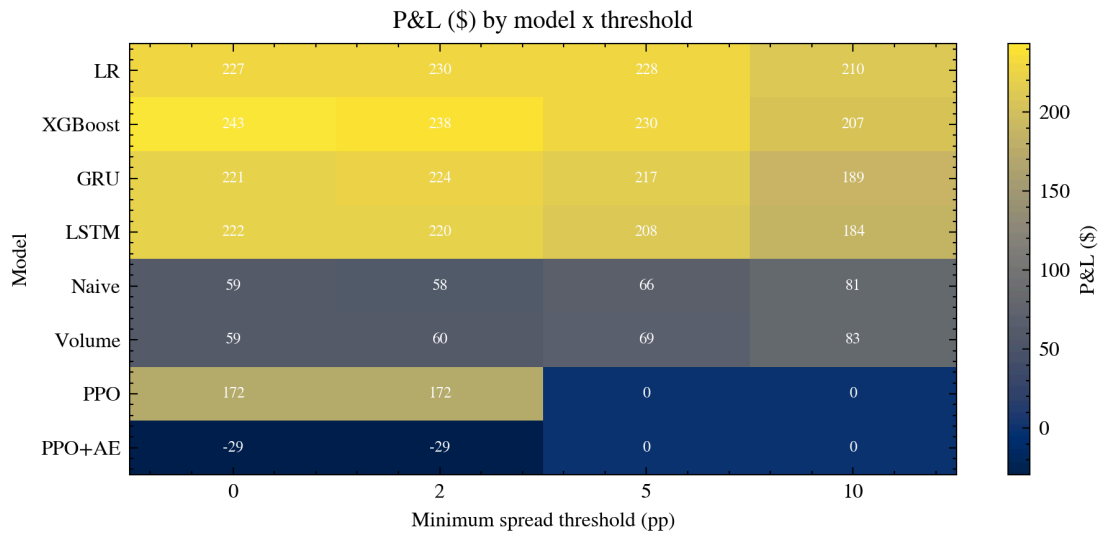


Figure 9. P&L heatmap by model $\times$ minimum-spread threshold (§5.13 / Proposal Experiment 3). Threshold = 2 pp dominates for all ML models; lower thresholds are dominated by noise, higher thresholds by missed trades.

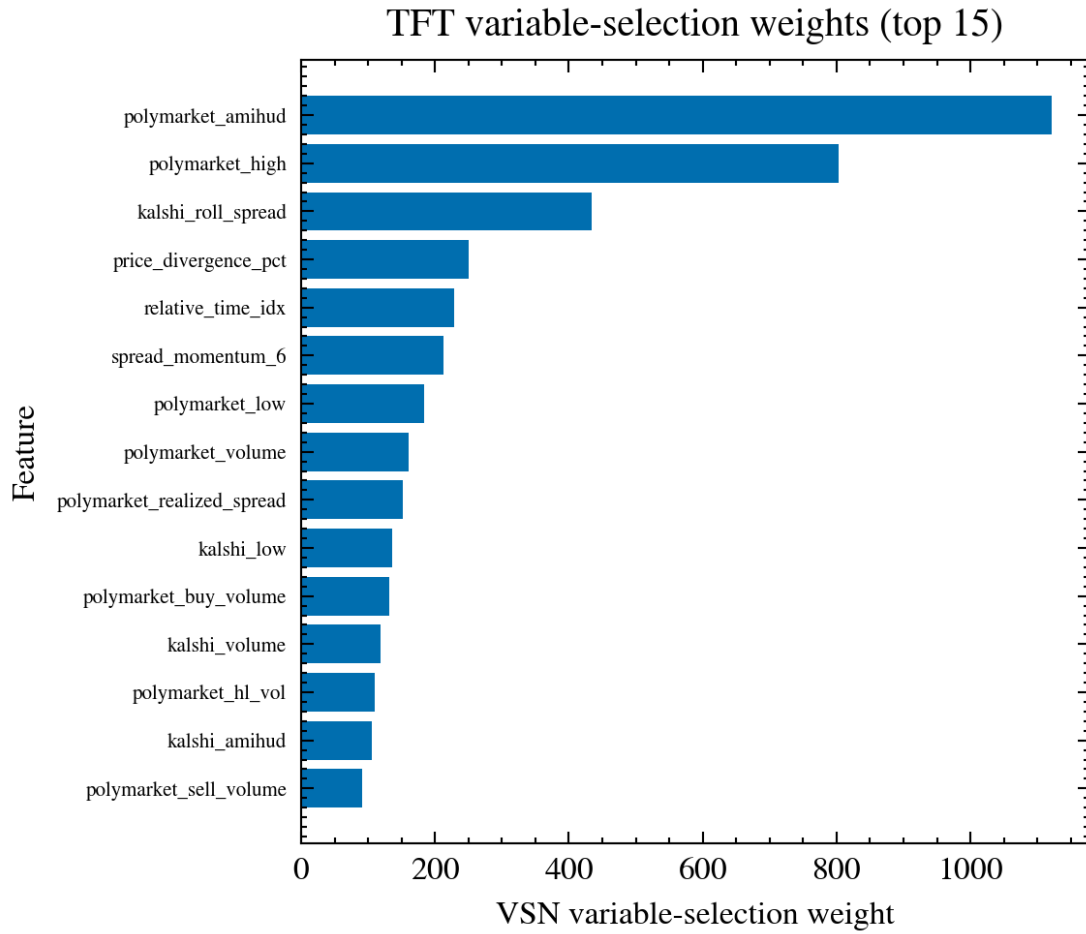


Figure 10. TFT VSN variable-selection weights (§6.2.3). The attention is healthy (entropy = 2.656 vs. degenerate baseline of ≈ 0) — TFT's underperformance vs. GRU is a data-volume bottleneck at $N=6,802$, not an architectural failure.

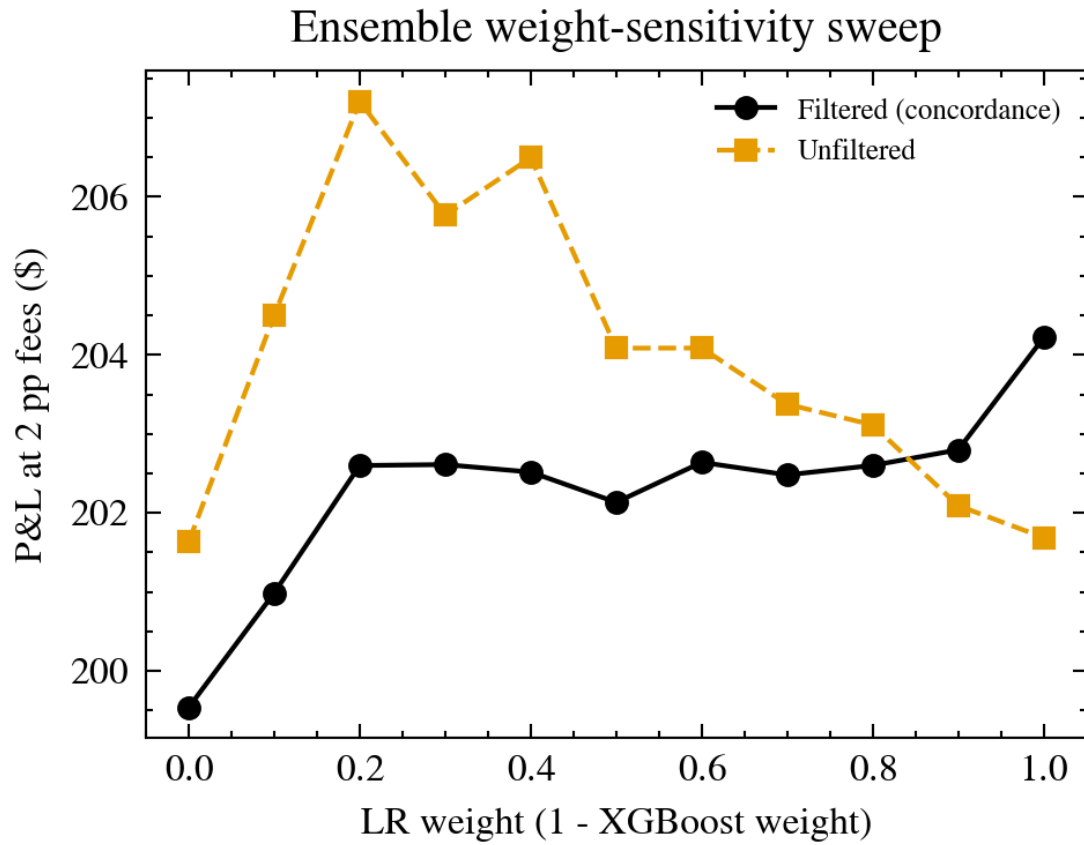


Figure 11. Walk-forward P&L at 2 pp fees across the 11-point LR-weight sweep (§5.11). The spread across weights is \$4.68; the curve is essentially flat, confirming the production ensemble's weight choice is not cherry-picked.